

Contents

1	Relatório do Sistema AQStore	16
1.0.1	Curso: Engenharia Informática	16
1.0.2	Instituição: Universidade de Santiago	16
1.0.3	Unidade Curricular: Conceção e Análise de Algoritmos	16
1.0.4	Tema: Sistema de Gestão Comercial AQStore	16
1.1	Participantes	16
1.2	Docente	17
2	Índice	17
3	1. Introducao	17
3.1	Enquadramento do Projeto	18
3.2	Justificação do Projeto	18
3.3	Metodologia Utilizada	18
3.4	Estrutura do Relatório	19
4	2. Objetivos	19
5	3. Arquitetura do Sistema	20
5.1	Interface Gráfica	20
5.2	Lógica de Negócio	21
5.3	Persistência de Dados	21
5.4	Documentação e Relatórios	21
5.5	Diagrama Simplificado	22
6	4. Estrutura do Projeto	22
6.1	Descrição das Classes	23
6.1.1	LoginApp.java	23
6.1.2	AQStoreMainFX.java	23
6.1.3	AQStoreSistema.java	23
6.1.4	Autenticacao.java	23
6.1.5	Usuario.java	23
6.1.6	Conexao.java	23
6.2	ReciboVenda.java	24
6.2.1	VisualizadorPDFFX.java	24
6.2.2	Organização Modular	24
7	5. Autenticacao e Gestao de Utilizadores	24
7.1	Objetivo da Autenticação	24
7.2	Níveis de Acesso Implementados	24
7.3	Classe Usuario	25
7.3.1	Código	25
7.3.2	Explicação	25
7.4	Classe Autenticacao	26
7.4.1	Consulta SQL Utilizada	26

7.4.2	Explicação	26
7.5	Validação das Credenciais	26
7.5.1	Código	26
7.5.2	Explicação	26
7.6	Execução da Consulta	26
7.6.1	Código	26
7.6.2	Explicação	27
7.7	Interface de Login	27
7.7.1	Código	27
7.7.2	Explicação	27
7.8	Processamento do Login	28
7.8.1	Código	28
7.8.2	Explicação	28
7.9	Validação de Campos Vazios	28
7.9.1	Código	28
7.9.2	Explicação	28
7.10	Login Bem-Sucedido	29
7.10.1	Código	29
7.10.2	Explicação	29
7.11	Login Inválido	29
7.11.1	Código	29
7.11.2	Explicação	29
7.12	Segurança Implementada	29
8	6. Conexao com a Base de Dados	30
8.1	Objetivo da Classe Conexao	30
8.2	Importação das Bibliotecas JDBC	30
8.2.1	Código	30
8.2.2	Explicação	31
8.3	Definição dos Parâmetros de Conexão	31
8.3.1	Código	31
8.3.2	Explicação	31
8.4	Construção da URL JDBC	32
8.4.1	Código	32
8.4.2	Explicação	32
8.4.3	Parâmetros Utilizados	32
8.5	Método de Conexão	33
8.5.1	Código	33
8.5.2	Explicação	33
8.6	Carregamento do Driver JDBC	33
8.6.1	Código	33
8.6.2	Explicação	33
8.7	Criação da Ligação	33
8.7.1	Código	33
8.7.2	Explicação	34
8.8	Tratamento de Erros	34

8.8.1	Driver Não Encontrado	34
8.8.2	Explicação	34
8.8.3	Erro de Ligação	34
8.8.4	Explicação	34
8.9	Exemplo de Utilização	35
8.9.1	Código	35
8.9.2	Explicação	35
8.10	Vantagens da Classe Conexao	35
8.10.1	Centralização	35
8.10.2	Reutilização	35
8.10.3	Facilidade de Manutenção	35
8.10.4	Organização	36
8.11	Fluxo da Conexão	36
9	7. Gestao de Produtos (CRUD)	36
9.1	Objetivos do Módulo	37
9.2	Estrutura da Tabela Produtos	37
9.3	CREATE – Inserção de Produtos	37
9.4	Código	37
9.5	Explicação	38
9.6	Declaração do Método	38
9.7	Consulta SQL	38
9.7.1	Explicação	38
9.8	Ligação à Base de Dados	39
9.8.1	Explicação	39
9.9	Criação do PreparedStatement	39
9.9.1	Explicação	39
9.10	Associação dos Parâmetros	39
9.10.1	Nome	39
9.10.2	Preço	40
9.10.3	Stock	40
9.11	Execução da Consulta	40
9.11.1	Explicação	40
9.12	Tratamento de Exceções	40
9.12.1	Explicação	41
9.13	Impressão do Erro	41
9.14	Retorno em Caso de Erro	41
9.15	Fluxograma do Método	41
9.16	Exemplo de Funcionamento	42
9.17	READ – Consulta de Produtos	42
9.18	Código	42
9.19	Explicação	43
9.20	Estrutura dos Dados	43
9.21	UPDATE – Atualização de Produtos	44
9.22	Código	44
9.23	Explicação	44

9.24	Exemplo	44
9.25	DELETE – Eliminação de Produtos	45
9.26	Código	45
9.27	Explicação	45
9.28	Exemplo	45
9.29	Interface Gráfica da Gestão de Produtos	46
9.30	Componentes Utilizados	46
9.31	Fluxo de Funcionamento	46
9.32	Segurança Implementada	46
9.33	Vantagens da Implementação CRUD	47
9.33.1	Organização	47
9.33.2	Facilidade de Atualização	47
9.33.3	Escalabilidade	47
9.33.4	Integração	47
10	8. Gestao de Vendas	47
10.1	Objetivos do Módulo	48
10.2	Estrutura de Dados Utilizada	48
10.3	Código	48
10.4	Explicação	48
10.5	Estrutura do Carrinho	48
10.6	Exemplo	49
10.7	Adição de Produtos ao Carrinho	49
10.8	Processo	49
10.9	Cálculo do Total	49
10.9.1	Exemplo	49
10.10	Formas de Pagamento	49
10.11	Opções Disponíveis	49
10.12	Pagamento em Dinheiro	50
10.13	Fórmula Utilizada	50
10.13.1	Exemplo	50
10.14	Finalização da Venda	50
10.15	Objetivos	50
10.16	Método Principal	50
10.17	Parâmetros	51
10.18	Registo da Venda	51
10.19	Estrutura da Tabela	51
10.20	Exemplo	51
10.21	Registo dos Produtos Vendidos	51
10.22	Estrutura	51
10.23	Exemplo	52
10.24	Transações na Base de Dados	52
10.25	Funcionamento	52
10.26	Commit	52
10.27	Rollback	52
10.28	Geração Automática do Recibo	53

10.29	Informações do Recibo	53
10.30	Pré-visualização do Recibo	53
10.31	Benefícios	53
10.32	Fluxo Completo da Venda	53
10.33	Vantagens da Implementação	54
10.33.1	Rapidez	54
10.33.2	Precisão	54
10.33.3	Segurança	54
10.33.4	Organização	54
10.33.5	Integração	54
11	9. Controlo Automatico de Stock	54
11.1	Verificar Stock	54
11.2	Atualizar Stock	55
11.3	Código	55
11.4	Explicação	55
12	10. Compras do Dia	55
12.1	Objetivos do Módulo	56
12.2	Interface Gráfica	56
12.3	Componentes Utilizados	56
12.4	Pesquisa por Data	56
12.5	Exemplo	56
12.6	Fluxo da Consulta	56
12.7	Consulta na Base de Dados	57
12.8	Objetivo	57
12.9	Informações Apresentadas	57
12.10	Exemplo de Resultado	57
12.11	Estrutura dos Dados	58
12.12	Campos Utilizados	58
12.13	Utilização da TableView	58
12.14	Vantagens	58
12.15	Eliminação de Vendas	58
12.16	Permissões	58
12.17	Controlo de Acesso	59
12.18	Lógica Aplicada	59
12.19	Justificação da Restrição	59
12.20	Processo de Eliminação	59
12.21	Atualização Automática	60
12.22	Benefícios do Módulo	60
12.23	Controlo Diário	60
12.24	Auditoria	60
12.25	Correção de Erros	60
12.26	Organização	60
12.27	Fluxo Completo	60
12.28	Integração com Outros Módulos	61

13 11. Relatorios	61
13.1 Objetivos do Módulo	61
13.2 Critérios de Pesquisa	62
13.3 Ano	62
13.3.1 Exemplo	62
13.4 Mês	62
13.4.1 Exemplo	62
13.5 Consulta por Ano e Mês	62
13.5.1 Exemplo	62
13.6 Fluxo de Funcionamento	62
13.7 Informações Apresentadas	63
13.8 Dados Disponíveis	63
13.9 Exemplo de Resultado	63
13.10Estrutura dos Dados	63
13.11Tabela Venda	63
13.12Tabela Compra	64
13.13Apresentação dos Relatórios	64
13.14Vantagens	64
13.15Exportação para CSV	64
13.16Objetivo	64
13.17Estrutura do Ficheiro	64
13.18Benefícios	64
13.19Exportação para Excel	65
13.20Biblioteca Utilizada	65
13.21Funcionalidades	65
13.22Cálculo de Totais	65
13.23Quantidade Total Vendida	65
13.24Valor Total Faturado	65
13.25Distribuição por Forma de Pagamento	65
13.26Benefícios para a Gestão	65
13.27Controlo Financeiro	65
13.28Apoio à Decisão	65
13.29Planeamento	66
13.30Integração com Outros Módulos	66
13.31Exemplo Prático	66
13.32Vantagens da Implementação	66
13.32.1 Automatização	67
13.32.2 Rapidez	67
13.32.3 Precisão	67
13.32.4 Flexibilidade	67
13.32.5 Exportação	67
14 12. Geracao de Recibos PDF	67
14.1 Objetivos do Módulo	68
14.2 Biblioteca Utilizada	68
14.3 Estrutura da Classe ReciboVenda	68

14.4 Pasta de Armazenamento	68
14.5 Código	68
14.6 Explicação	68
14.7 Criação da Pasta	69
14.8 Código	69
14.9 Explicação	69
14.10 Geração do Nome do Ficheiro	69
14.11 Código	69
14.12 Explicação	69
14.13 Criação do Documento PDF	69
14.14 Código	69
14.15 Explicação	70
14.16 Abertura do Documento	70
14.17 Código	70
14.18 Explicação	70
14.19 Definição de Fontes	70
14.20 Código	70
14.21 Explicação	71
14.21.1 Fonte Título	71
14.21.2 Fonte Normal	71
14.22 Cabeçalho do Recibo	71
14.23 Código	71
14.24 Explicação	71
14.25 Informações da Venda	71
14.26 Código	71
14.27 Explicação	72
14.28 Criação da Tabela de Produtos	72
14.29 Código	72
14.30 Explicação	72
14.31 Cabeçalhos da Tabela	72
14.32 Código	72
14.33 Explicação	73
14.34 Inserção dos Produtos	73
14.35 Código	73
14.36 Explicação	73
14.37 Cálculo do Total Geral	74
14.38 Código	74
14.39 Explicação	74
14.40 Exemplo	74
14.41 Valor Recebido e Troco	74
14.42 Código	74
14.43 Explicação	74
14.44 Mensagem Final	75
14.45 Código	75
14.46 Explicação	75
14.47 Fecho do Documento	75

14.48	Código	75
14.49	Explicação	75
14.50	Exemplo de Recibo	75
14.51	Benefícios da Implementação	76
14.51.1	Rapidez	76
14.51.2	Organização	76
14.51.3	Segurança	76
14.51.4	Profissionalismo	76
14.51.5	Integração	76
14.52	Fluxo Completo	76
15	13. Pre-visualizacao de Recibos	77
15.1	Objetivos do Módulo	77
15.2	Biblioteca Utilizada	77
15.3	Classe Responsável	77
15.4	Abertura do Documento PDF	78
15.5	Código	78
15.6	Explicação	78
15.7	Exemplo	78
15.8	Criação do Renderizador	78
15.9	Código	78
15.10	Explicação	78
15.11	Conversão da Página em Imagem	79
15.12	Código	79
15.13	Explicação	79
15.13.1	Parâmetros	79
15.14	Resultado	79
15.15	Conversão para JavaFX	79
15.16	Código	80
15.17	Explicação	80
15.18	Criação do ImageView	80
15.19	Código	80
15.20	Explicação	80
15.21	Ajuste Automático da Imagem	80
15.22	Código	80
15.23	Explicação	81
15.23.1	Preserve Ratio	81
15.23.2	Fit Width	81
15.24	Utilização de ScrollPane	81
15.25	Código	81
15.26	Explicação	81
15.27	Benefícios	81
15.28	Criação da Janela	81
15.29	Código	81
15.30	Explicação	82
15.31	Configuração do Stage	82

15.32	Código	82
15.33	Explicação	82
15.34	Exibição da Janela	82
15.35	Código	82
15.36	Explicação	83
15.37	Fluxo Completo da Pré-visualização	83
15.38	Benefícios da Implementação	83
15.39	Integração	83
15.40	Rapidez	83
15.41	Facilidade de Utilização	83
15.42	Melhor Experiência do Utilizador	84
15.43	Organização	84
15.44	Exemplo de Funcionamento	84
16	14. Interface Grafica	84
16.1	Objetivos da Interface	85
16.2	Estrutura Geral da Interface	85
16.3	Menu Lateral	85
16.4	Área de Conteúdo	85
16.5	Estrutura Visual	85
16.6	Janela de Login	85
16.7	Código	85
16.8	Explicação	86
16.9	Botão de Login	86
16.10	Código	86
16.11	Explicação	86
16.12	Janela Principal	86
16.13	Código	86
16.14	Explicação	87
16.15	Menu Lateral	87
16.16	Código	87
16.17	Explicação	87
16.17.1	Loja / Caixa	88
16.17.2	Produtos	88
16.17.3	Compras do Dia	88
16.17.4	Relatórios	88
16.18	Controlo de Permissões na Interface	88
16.19	Código	88
16.20	Explicação	88
16.21	Utilização do BorderPane	88
16.22	Explicação	88
16.23	Utilização de VBox	89
16.24	Código	89
16.25	Explicação	89
16.26	Utilização de HBox	89
16.27	Explicação	89

16.28	Utilização de TableView	89
16.29	Código	90
16.30	Explicação	90
16.31	Exemplo de Utilização	90
16.32	Utilização de Alert	90
16.33	Código	90
16.34	Explicação	90
16.35	Exemplo	90
16.36	Interface do Carrinho	91
16.37	Estrutura	91
16.38	Interface dos Relatórios	91
16.39	Interface da Pré-visualização PDF	91
16.40	Código	91
16.41	Explicação	92
16.42	Benefícios da Interface Implementada	92
16.43	Facilidade de Utilização	92
16.44	Organização	92
16.45	Rapidez	92
16.46	Segurança	92
16.47	Integração	92
16.48	Fluxo de Navegação	92
17	15. Implementacao do CRUD	93
17.1	Objetivos da Implementação CRUD	93
17.2	CREATE – Inserção de Dados	94
17.3	Exemplo: Inserção de Produtos	94
17.3.1	Código	94
17.4	Explicação	94
17.5	Exemplo Prático	95
17.6	READ – Consulta de Dados	95
17.7	Exemplo: Consulta de Produtos	95
17.7.1	Código	95
17.8	Explicação	95
17.9	Processamento dos Resultados	96
17.9.1	Código	96
17.10	Explicação	96
17.11	Resultado	96
17.12	UPDATE – Atualização de Dados	96
17.13	Exemplo: Atualizar Produto	97
17.13.1	Código	97
17.14	Explicação	97
17.15	Exemplo	97
17.16	DELETE – Eliminação de Dados	97
17.17	Exemplo: Eliminar Produto	98
17.17.1	Código	98
17.18	Explicação	98

17.19	Exemplo	98
17.20	Utilização do PreparedStatement	98
17.21	Código	98
17.22	Vantagens	99
17.22.1	Segurança	99
17.22.2	Desempenho	99
17.22.3	Organização	99
17.23	Integração com a Interface JavaFX	99
17.24	Inserir	99
17.25	Consultar	99
17.26	Atualizar	99
17.27	Eliminar	99
17.28	Fluxo de Funcionamento	99
17.29	Benefícios da Implementação CRUD	100
17.30	Organização	100
17.31	Facilidade de Manutenção	100
17.32	Escalabilidade	100
17.33	Integração	100
17.34	Segurança	100
17.35	Aplicação do CRUD nos Módulos	100
17.36	Exemplo Completo	100
17.37	Produto	100
17.37.1	Inserir	100
17.37.2	Consultar	101
17.37.3	Atualizar	101
17.37.4	Eliminar	101
18	16. Estruturas de Dados Utilizadas	101
18.1	Conceito de Estrutura de Dados	102
18.2	Estrutura Principal Utilizada	102
18.3	Implementação da Lista Dinâmica	102
18.4	Código	102
18.5	Explicação	102
18.6	Funcionamento da Lista	102
18.7	Estrutura	102
18.8	Organização dos Dados	103
18.9	Exemplo	103
18.10	Inserção de Elementos	103
18.11	Código	103
18.12	Explicação	104
18.13	Resultado	104
18.14	Percurso da Lista	104
18.15	Código	104
18.16	Explicação	104
18.17	Exemplo	104
18.18	Acesso aos Dados	104

18.19	Código	105
18.20	Explicação	105
18.21	Estruturas Utilizadas nos Relatórios	105
18.22	Código	105
18.23	Explicação	105
18.24	Estruturas Utilizadas na Consulta de Produtos	105
18.25	Código	105
18.26	Explicação	106
18.27	Integração com JavaFX	106
18.28	Exemplo	106
18.29	Explicação	106
18.30	Porque Não Foi Utilizada uma Pilha?	106
18.31	Exemplo de Pilha	106
18.32	Porque Não Foi Utilizada uma Fila?	106
18.33	Exemplo de Fila	107
18.34	Comparação das Estruturas	107
18.35	Justificação da Escolha	107
18.36	Flexibilidade	107
18.37	Simplicidade	107
18.38	Integração	107
18.39	Desempenho	108
18.40	Organização	108
18.41	Fluxo de Utilização	108
18.42	Vantagens da Estrutura Escolhida	108
18.42.1	Facilidade de Programação	108
18.42.2	Escalabilidade	108
18.42.3	Manutenção	108
18.42.4	Integração com Base de Dados	108
18.42.5	Integração com Interface Gráfica	109
18.43	Exemplo Prático	109
19	17. Controlo de Permissoes	110
19.1	Objetivos do Controlo de Permissões	110
19.2	Níveis de Acesso Implementados	110
19.3	Perfil ADMIN	110
19.4	Funcionalidades Disponíveis	110
19.5	Responsabilidades	111
19.6	Perfil GESTOR	111
19.7	Funcionalidades Disponíveis	111
19.8	Restrições	111
19.9	Perfil ATENDENTE	111
19.10	Funcionalidades Disponíveis	111
19.11	Restrições	112
19.12	Armazenamento das Permissões	112
19.13	Estrutura	112
19.14	Exemplo	112

19.15	Identificação do Perfil	112
19.16	Código	112
19.17	Explicação	113
19.18	Verificação de Permissões	113
19.19	Código	113
19.20	Explicação	113
19.21	Verificação do Perfil Gestor	113
19.22	Código	113
19.23	Explicação	114
19.24	Utilização das Permissões	114
19.25	Exemplo: Gestão de Utilizadores	114
19.25.1	Código	114
19.25.2	Explicação	114
19.26	Exemplo: Compras do Dia	114
19.27	Código	114
19.28	Explicação	114
19.29	Proteção Adicional	115
19.30	Código	115
19.31	Explicação	115
19.32	Fluxo de Verificação	115
19.33	Exemplo de Funcionamento	116
19.34	Utilizador ADMIN	116
19.35	Utilizador GESTOR	116
19.36	Utilizador ATENDENTE	116
19.37	Benefícios do Controlo de Permissões	116
19.38	Segurança	116
19.39	Organização	117
19.40	Integridade	117
19.41	Facilidade de Gestão	117
19.42	Auditoria	117
19.43	Integração com Outros Módulos	117
20	18. Tecnologias Utilizadas	118
20.1	Linguagem de Programação	118
20.2	Java	118
20.2.1	Características	118
20.2.2	Aplicação no Projeto	118
20.3	Exemplo de Código	118
20.3.1	Explicação	119
20.4	Interface Gráfica	119
20.5	JavaFX	119
20.6	Funcionalidades Utilizadas	119
20.7	Exemplo	119
20.8	Exemplo	119
20.9	Benefícios	119
20.9.1	Interface Moderna	119

20.9.2	Integração	119
20.9.3	Produtividade	120
20.10	Base de Dados	120
20.11	MySQL	120
20.12	Dados Armazenados	120
20.13	Estrutura da Base de Dados	120
20.14	Exemplo de Consulta	120
20.15	Benefícios	120
20.15.1	Segurança	120
20.15.2	Organização	120
20.15.3	Escalabilidade	120
20.16	JDBC	120
20.17	Java Database Connectivity	120
20.18	Objetivo	121
20.19	Exemplo	121
20.20	Explicação	121
20.21	OpenPDF	121
20.22	Biblioteca de Geração de PDFs	121
20.23	Funcionalidades	121
20.24	Exemplo	121
20.25	Aplicação	122
20.26	Apache PDFBox	122
20.27	Biblioteca de Visualização PDF	122
20.28	Funcionalidades	122
20.29	Exemplo	122
20.30	Aplicação	122
20.31	Apache POI	122
20.32	Biblioteca para Excel	122
20.33	Funcionalidades	122
20.34	Exemplo	123
20.35	Aplicação	123
20.36	CSV	123
20.37	Exportação de Dados	123
20.38	Exemplo	123
20.39	Benefícios	123
20.40	Maven	123
20.41	Gestão de Dependências	123
20.42	Benefícios	123
20.42.1	Automatização	123
20.42.2	Organização	124
20.43	Exemplo	124
20.44	IDE Utilizada	124
20.45	Visual Studio Code	124
20.46	Recursos Utilizados	124
20.47	Git e GitHub	124
20.48	Controlo de Versões	124

20.49	Benefícios	124
20.50	Comandos Utilizados	125
20.51	Sistema Operativo	125
20.52	Windows 10/11	125
20.53	Tecnologias Integradas	125
20.54	Tabela Resumo	125
20.55	Benefícios da Arquitetura Tecnológica	126
20.55.1	Modularidade	126
20.55.2	Escalabilidade	126
20.55.3	Manutenção	126
20.55.4	Segurança	126
20.55.5	Produtividade	126
21	19. Resultados Obtidos	127
21.1	Funcionalidades Implementadas	127
21.2	Autenticação de Utilizadores	127
21.2.1	Resultado	127
21.3	Gestão de Produtos	127
21.3.1	Funcionalidades	127
21.3.2	Resultado	128
21.4	Gestão de Vendas	128
21.4.1	Resultado	128
21.5	Compras do Dia	128
21.5.1	Funcionalidades	128
21.5.2	Resultado	128
21.6	Relatórios	128
21.6.1	Funcionalidades	128
21.6.2	Resultado	128
21.7	Recibos PDF	129
21.7.1	Funcionalidades	129
21.7.2	Resultado	129
21.8	Pré-visualização de Recibos	129
21.8.1	Funcionalidades	129
21.8.2	Resultado	129
21.9	Integração dos Módulos	129
21.10	Fluxo Implementado	129
21.11	Resultados na Base de Dados	130
21.12	Operações Testadas	130
21.12.1	Resultado	130
21.13	Resultados da Interface Gráfica	130
21.14	Características Observadas	130
21.14.1	Resultado	130
21.15	Resultados do Controlo de Permissões	130
21.16	Perfis Testados	131
21.16.1	ADMIN	131
21.16.2	GESTOR	131

21.16.3 ATENDENTE	131
21.16.4 Resultado	131
21.17 Resultados da Exportação	131
21.18 CSV	131
21.19 Excel	131
21.20 Compatibilidade	131
21.21 Resultados dos Recibos PDF	132
21.22 Informações Impressas	132
21.22.1 Resultado	132
21.23 Resultados da Pré-visualização	132
21.23.1 Observações	132
21.23.2 Resultado	132
21.24 Desempenho do Sistema	132
21.25 Avaliação	132
21.26 Benefícios Obtidos	133
21.27 Automação	133
21.28 Organização	133
21.29 Segurança	133
21.30 Rapidez	133
21.31 Fiabilidade	133
21.32 Aplicação dos Conhecimentos Académicos	133
21.33 Programação	133
21.34 Bases de Dados	133
21.35 Conceção e Análise de Algoritmos	133
21.36 Interfaces Gráficas	134
21.37 Avaliação Geral dos Resultados	134
21.37.1 Funcionalidades Implementadas	134
22 20. Conclusao	134

1 Relatório do Sistema AQStore

1.0.1 Curso: Engenharia Informática

1.0.2 Instituição: Universidade de Santiago

1.0.3 Unidade Curricular: Conceção e Análise de Algoritmos

1.0.4 Tema: Sistema de Gestão Comercial AQStore

1.1 Participantes

- Paulo Costa Nº 7517
- Kévin Guiomar Nº 7543
- Claudia Martins Nº 7751

1.2 Docente

- Professor Valério Semedo

2 Índice

1. Introducao
2. Objetivos
3. Arquitetura do Sistema
4. Estrutura do Projeto
5. Autenticacao e Gestao de Utilizadores
6. Conexao com a Base de Dados
7. Gestao de Produtos (CRUD)
8. Gestao de Vendas
9. Controlo Automatico de Stock
10. Compras do Dia
11. Relatorios
12. Geracao de Recibos PDF
13. Pre-visualizacao de Recibos
14. Interface Grafica
15. Implementação do CRUD
16. Estruturas de Dados Utilizadas
17. Controlo de Permissões
18. Tecnologias Utilizadas
19. Resultados Obtidos
20. Conclusão

3 1. Introducao

O sistema **AQStore – Loja Aquapet Lda** foi desenvolvido com o objetivo de apoiar a gestão comercial de uma loja de produtos para animais. A aplicação permite gerir produtos, utilizadores, vendas, relatórios, recibos em PDF e controlo de stock.

O projeto foi desenvolvido em **Java**, utilizando **JavaFX** para a interface gráfica, **MySQL** para armazenamento dos dados, **JDBC** para ligação à base de dados, **OpenPDF** para geração de recibos e **PDFBox** para pré-visualização dos documentos.

A aplicação implementa funcionalidades fundamentais para um sistema de gestão comercial, incluindo:

- Autenticação de utilizadores;
- Controlo de permissões por nível de acesso;
- Gestão de produtos através de operações CRUD;
- Registo e processamento de vendas;

- Consulta de compras realizadas;
- Geração de relatórios por ano e mês;
- Exportação de dados para CSV e Excel;
- Emissão automática de recibos PDF;
- Pré-visualização dos recibos diretamente na aplicação.

3.1 Enquadramento do Projeto

Atualmente, muitas pequenas e médias empresas ainda recorrem a métodos manuais ou ferramentas limitadas para efetuar o controlo das suas vendas e produtos. Esta situação pode originar dificuldades na gestão da informação, erros de registo e perda de produtividade.

Com o objetivo de minimizar estes problemas, surgiu a necessidade de desenvolver uma aplicação que permitisse automatizar as principais tarefas relacionadas com a atividade comercial da loja, oferecendo uma plataforma simples, segura e eficiente para a gestão diária das operações.

O AQStore foi desenvolvido precisamente para responder a essa necessidade, disponibilizando funcionalidades que permitem centralizar informações, controlar acessos, emitir recibos e gerar relatórios de forma rápida e organizada.

3.2 Justificação do Projeto

A escolha deste projeto deve-se à sua relevância prática e à possibilidade de integrar diversos conteúdos estudados ao longo do curso.

O desenvolvimento do sistema permitiu aplicar conhecimentos relacionados com:

- Programação Java;
- Programação Orientada a Objetos;
- Bases de Dados MySQL;
- JDBC;
- Interfaces Gráficas JavaFX;
- Estruturas de Dados;
- Gestão de Utilizadores;
- Geração de Documentos PDF;
- Exportação de Dados.

Além disso, o projeto proporcionou uma experiência próxima de um ambiente empresarial real, permitindo compreender os desafios associados ao desenvolvimento de aplicações de gestão.

3.3 Metodologia Utilizada

Para o desenvolvimento do sistema foram seguidas as seguintes etapas:

1. Levantamento dos requisitos;
2. Modelação da solução;

3. Criação da base de dados;
4. Desenvolvimento da interface gráfica;
5. Implementação das regras de negócio;
6. Integração com a base de dados;
7. Implementação dos relatórios;
8. Geração de recibos PDF;
9. Testes e validações;
10. Documentação do projeto.

Cada etapa contribuiu para a construção gradual do sistema, permitindo obter uma solução organizada, modular e de fácil manutenção.

3.4 Estrutura do Relatório

O presente relatório encontra-se organizado em vários capítulos, abordando os diferentes aspetos do projeto, desde a sua conceção até aos resultados obtidos.

São apresentados:

- Objetivos do projeto;
- Arquitetura do sistema;
- Estrutura do projeto;
- Autenticação e gestão de utilizadores;
- Conexão com a base de dados;
- Implementação do CRUD;
- Gestão de vendas;
- Relatórios;
- Geração e pré-visualização de recibos;
- Estruturas de dados utilizadas;
- Tecnologias empregues;
- Resultados obtidos;
- Conclusões e propostas de melhoria.

Esta organização permite uma visão completa do desenvolvimento do sistema AQStore, evidenciando as principais decisões técnicas e os resultados alcançados.

4 2. Objetivos

O desenvolvimento do sistema AQStore teve como principal objetivo criar uma aplicação de gestão comercial para a Loja Aquapet Lda, permitindo controlar produtos, vendas, utilizadores e relatórios através de uma interface gráfica intuitiva.

Os objetivos específicos do projeto foram:

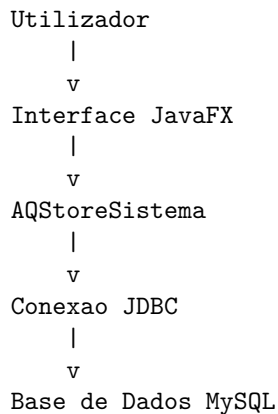
- Desenvolver uma aplicação desktop utilizando JavaFX;
- Implementar autenticação de utilizadores;
- Criar diferentes níveis de acesso (ADMIN, GESTOR e ATENDENTE);

- Implementar operações CRUD para gestão de produtos;
- Registrar vendas e respetivos produtos vendidos;
- Gerar recibos em formato PDF;
- Disponibilizar pré-visualização dos recibos dentro da aplicação;
- Implementar consultas de compras por data;
- Criar relatórios de vendas por ano e mês;
- Exportar relatórios para CSV e Excel;
- Integrar a aplicação com uma base de dados MySQL;
- Aplicar boas práticas de Programação Orientada a Objetos.

5 3. Arquitetura do Sistema

O AQStore foi desenvolvido seguindo uma arquitetura modular, separando as responsabilidades em diferentes classes para facilitar a manutenção, organização e reutilização do código.

A arquitetura do sistema é composta por quatro camadas principais:



A interface gráfica recebe os dados do utilizador, a classe AQStoreSistema processa as regras de negócio e a classe Conexao estabelece a comunicação com o MySQL.

5.1 Interface Gráfica

Responsável pela interação com o utilizador.

Classes:

- LoginApp
- AQStoreMainFX
- VisualizadorPDFFX

Funções:

- Receber dados do utilizador;

- Apresentar menus;
- Exibir tabelas e relatórios;
- Mostrar mensagens e alertas;
- Apresentar recibos PDF.

5.2 Lógica de Negócio

Responsável pelo processamento das regras do sistema.

Classes:

- AQStoreSistema
- Autenticacao

Funções:

- Validar utilizadores;
- Processar vendas;
- Gerir produtos;
- Gerir utilizadores;
- Gerar relatórios.

5.3 Persistência de Dados

Responsável pela comunicação com a base de dados MySQL.

Classe:

- Conexao

Funções:

- Abrir ligações JDBC;
- Executar consultas SQL;
- Inserir, atualizar e eliminar dados.

5.4 Documentação e Relatórios

Responsável pela criação e visualização de documentos.

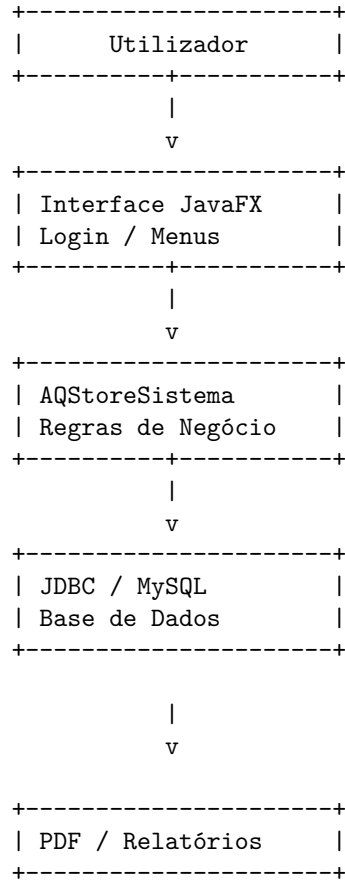
Classes:

- ReciboVenda
- VisualizadorPDFFX

Funções:

- Gerar recibos PDF;
- Visualizar recibos;
- Exportar relatórios.

5.5 Diagrama Simplificado



6 4. Estrutura do Projeto

O projeto encontra-se organizado em diferentes classes, cada uma responsável por uma funcionalidade específica.

Estrutura Geral

AQStore

```
LoginApp.java
AQStoreMainFX.java
AQStoreSistema.java
Autenticacao.java
Usuario.java
Conexao.java
```

```
ReciboVenda.java
VisualizadorPDFFX.java
```

```
recibos/
```

```
resources/
    icon.png
```

6.1 Descrição das Classes

6.1.1 LoginApp.java

Responsável pelo ecrã de autenticação.

Principais funções: - Receber credenciais; - Validar login; - Abrir a janela principal; - Apresentar mensagens de erro.

6.1.2 AQStoreMainFX.java

Classe principal da interface gráfica.

Principais funções: - Construção dos menus; - Gestão das páginas; - Navegação entre funcionalidades; - Controlo de permissões.

6.1.3 AQStoreSistema.java

Responsável pelas regras de negócio.

Principais funções: - Gestão de produtos; - Gestão de utilizadores; - Registo de vendas; - Relatórios; - Compras do dia.

6.1.4 Autenticacao.java

Responsável pela validação dos utilizadores.

Principais funções: - Verificar credenciais; - Identificar o nível de acesso; - Criar objetos Usuario.

6.1.5 Usuario.java

Representa os utilizadores do sistema.

Atributos: - id - username - password - nivelAcesso

6.1.6 Conexao.java

Responsável pela ligação à base de dados.

Principais funções: - Abrir conexão JDBC; - Fornecer ligação às restantes classes.

6.2 ReciboVenda.java

Responsável pela geração dos recibos PDF.

Principais funções: - Criar documento PDF; - Inserir produtos; - Inserir totais; - Guardar recibos.

6.2.1 VisualizadorPDFFX.java

Responsável pela pré-visualização dos recibos.

Principais funções: - Abrir ficheiros PDF; - Converter PDF em imagem; - Apresentar numa janela JavaFX.

6.2.2 Organização Modular

A divisão do sistema em várias classes permitiu: - Melhor organização do código; - Facilidade de manutenção; - Reutilização de funcionalidades; - Maior escalabilidade; - Separação de responsabilidades.

Cada classe possui uma função específica, reduzindo a dependência entre módulos e facilitando futuras evoluções do sistema.

7 5. Autenticacao e Gestao de Utilizadores

A autenticação é uma das funcionalidades mais importantes do sistema AQ-Store, pois garante que apenas utilizadores autorizados conseguem aceder às funcionalidades da aplicação.

O sistema utiliza uma tabela denominada **usuarios**, armazenada na base de dados MySQL, onde são guardadas as credenciais de acesso e o respetivo nível de permissão de cada utilizador.

7.1 Objetivo da Autenticação

O módulo de autenticação tem como principais objetivos:

- Validar as credenciais dos utilizadores;
- Controlar o acesso às funcionalidades do sistema;
- Identificar o nível de acesso do utilizador;
- Garantir a segurança dos dados;
- Impedir acessos não autorizados.

7.2 Níveis de Acesso Implementados

O sistema implementa três níveis distintos de acesso:

Nível	Permissões
ADMIN	Acesso total ao sistema
GESTOR	Gestão de produtos, vendas e relatórios
ATENDENTE	Operações de caixa e consulta de informações

Após o login, o sistema identifica automaticamente o perfil do utilizador e disponibiliza apenas as funcionalidades permitidas para esse nível de acesso.

7.3 Classe Usuario

A classe `Usuario` é responsável por representar cada utilizador do sistema.

7.3.1 Código

```
public class Usuario {

    private int id;
    private String username;
    private String password;
    private String nivelAcesso;

    public Usuario(int id, String username,
                  String password,
                  String nivelAcesso) {

        this.id = id;
        this.username = username;
        this.password = password;
        this.nivelAcesso = nivelAcesso;
    }
}
```

7.3.2 Explicação

Esta classe funciona como um modelo de dados (Model), armazenando as informações do utilizador autenticado.

Os seus atributos possuem as seguintes funções:

Atributo	Descrição
id	Identificador único do utilizador
username	Nome de utilizador
password	Palavra-passe
nivelAcesso	Perfil de acesso do utilizador

Após o login, os dados do utilizador ficam disponíveis para o restante sistema.

7.4 Classe Autenticacao

A validação das credenciais é realizada através da classe `Autenticacao`.

7.4.1 Consulta SQL Utilizada

```
String sql =  
"SELECT id, username, password, nivel  
FROM usuarios  
WHERE username = ? AND password = ?";
```

7.4.2 Explicação

Esta consulta procura na tabela `usuarios` um registo que possua simultaneamente:

- o nome de utilizador indicado;
- a palavra-passe indicada.

Caso exista correspondência, o utilizador é considerado válido.

Os símbolos `?` representam parâmetros dinâmicos que serão preenchidos durante a execução da consulta.

A utilização de `PreparedStatement` aumenta a segurança da aplicação e reduz o risco de SQL Injection.

7.5 Validação das Credenciais

7.5.1 Código

```
stmt.setString(1, username.trim());  
stmt.setString(2, password.trim());
```

7.5.2 Explicação

Os dados introduzidos pelo utilizador são enviados para a consulta SQL.

O método `trim()` remove espaços em branco antes e depois do texto, evitando erros de autenticação causados por caracteres desnecessários.

7.6 Execução da Consulta

7.6.1 Código

```
try (ResultSet rs = stmt.executeQuery()) {  
  
    if (rs.next()) {
```

```

        return new Usuario(
            rs.getInt("id"),
            rs.getString("username"),
            rs.getString("password"),
            rs.getString("nivel")
        );
    }
}

```

7.6.2 Explicação

Após executar a consulta:

- Se existir um registo correspondente, é criado um objeto da classe `Usuario`;
- Os dados encontrados são carregados para memória;
- O objeto é devolvido à aplicação.

Caso não exista qualquer correspondência, o método devolve:

```
return null;
```

indicando que o login falhou.

7.7 Interface de Login

O ecrã de autenticação foi desenvolvido utilizando JavaFX.

7.7.1 Código

```

Label lblUsuario = new Label("Usuário:");
Label lblSenha = new Label("Senha:");

txtUsuario = new TextField();
txtSenha = new PasswordField();

Button btnEntrar = new Button("Entrar");

```

7.7.2 Explicação

A interface disponibiliza:

- Campo para introdução do utilizador;
- Campo para introdução da palavra-passe;
- Botão para iniciar o processo de autenticação.

O componente:

`PasswordField`

Ocultar automaticamente os caracteres introduzidos pelo utilizador, aumentando a segurança do sistema.

7.8 Processamento do Login

7.8.1 Código

```
btnEntrar.setOnAction(  
    e -> fazerLogin(stage)  
);
```

7.8.2 Explicação

Quando o utilizador pressiona o botão “Entrar”, é executado o método:

`fazerLogin()`

Este método:

1. Obtém os dados introduzidos;
2. Valida se os campos estão preenchidos;
3. Chama a classe `Autenticacao`;
4. Verifica o resultado;
5. Abre a janela principal do sistema.

7.9 Validação de Campos Vazios

7.9.1 Código

```
if (usuario.isEmpty() || senha.isEmpty()) {  
  
    mostrarAlerta(  
        Alert.AlertType.WARNING,  
        "Aviso",  
        "Preencha usuário e senha."  
    );  
  
    return;  
}
```

7.9.2 Explicação

Antes de efetuar a autenticação, o sistema verifica se os campos foram preenchidos.

Caso algum campo esteja vazio:

- o login é cancelado;
- é apresentada uma mensagem de aviso ao utilizador.

7.10 Login Bem-Sucedido

7.10.1 Código

```
if (u != null) {  
  
    AQStoreMainFX main =  
        new AQStoreMainFX(u);  
  
    Stage novoStage =  
        new Stage();  
  
    main.start(novoStage);  
  
    stageAtual.close();  
}
```

7.10.2 Explicação

Quando a autenticação é válida:

- É criada uma nova instância da janela principal;
- O utilizador autenticado é enviado para o sistema;
- A janela de login é encerrada;
- O utilizador passa a utilizar o AQStore.

7.11 Login Inválido

7.11.1 Código

```
mostrarAlerta(  
    Alert.AlertType.ERROR,  
    "Erro",  
    "Usuário ou senha inválidos."  
);
```

7.11.2 Explicação

Caso as credenciais não existam na base de dados:

- O acesso é recusado;
- É apresentada uma mensagem de erro;
- O utilizador permanece na janela de login.

7.12 Segurança Implementada

Durante a implementação da autenticação foram aplicadas diversas medidas de segurança:

- Utilização de PreparedStatement;
- Controlo de níveis de acesso;
- Validação de campos obrigatórios;
- Ocultação da palavra-passe;
- Tratamento de exceções;
- Restrição de funcionalidades por perfil.

Estas medidas contribuem para tornar o sistema mais robusto e seguro.

8 6. Conexao com a Base de Dados

A comunicação entre o sistema AQStore e a base de dados MySQL é realizada através da classe **Conexao**.

Esta classe é responsável por:

- Estabelecer a ligação com o servidor MySQL;
- Disponibilizar uma conexão para as restantes classes do sistema;
- Centralizar os parâmetros de acesso à base de dados;
- Tratar possíveis erros de conexão.

A utilização de uma classe específica para a conexão permite uma melhor organização do código e facilita futuras alterações na configuração do banco de dados.

8.1 Objetivo da Classe Conexao

A principal função desta classe é fornecer uma ligação ativa entre a aplicação Java e a base de dados MySQL.

Através desta ligação, o sistema consegue:

- Consultar dados;
- Inserir novos registos;
- Atualizar informações;
- Eliminar dados;
- Executar relatórios.

Sem esta ligação, nenhuma funcionalidade que depende da base de dados poderia funcionar.

8.2 Importação das Bibliotecas JDBC

8.2.1 Código

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
```

8.2.2 Explicação

Estas bibliotecas fazem parte da API JDBC (Java Database Connectivity).

Cada uma possui uma função específica:

Biblioteca	Função
Connection	Representa uma ligação à base de dados
DriverManager	Responsável por criar ligações
SQLException	Trata erros relacionados com SQL

Estas bibliotecas permitem que o Java comunique diretamente com o MySQL.

8.3 Definição dos Parâmetros de Conexão

8.3.1 Código

```
private static final String HOST = "localhost";
private static final String PORTA = "3306";
private static final String BANCO = "aqstore";
private static final String USER = "root";
private static final String PASSWORD = "++++++";
```

8.3.2 Explicação

Estas constantes armazenam os parâmetros necessários para aceder à base de dados.

8.3.2.1 HOST

```
private static final String HOST = "localhost";
```

Define o servidor onde a base de dados está instalada.

No caso do projeto:

localhost

Significa que o MySQL está instalado no mesmo computador da aplicação.

8.3.2.2 PORTA

```
private static final String PORTA = "3306";
```

A porta 3306 é a porta padrão utilizada pelo MySQL.

8.3.2.3 BANCO

```
private static final String BANCO = "aqstore";
```

Indica o nome da base de dados utilizada pelo sistema.

8.3.2.4 USER

```
private static final String USER = "root";
```

Define o utilizador que possui permissões para aceder à base de dados.

8.3.2.5 PASSWORD

```
private static final String PASSWORD = "++++++";
```

Define a palavra-passe utilizada para autenticação. Por segurança, este valor não deve ficar escrito diretamente no código; o ideal é usar variáveis de ambiente ou um ficheiro de configuração protegido.

8.4 Construção da URL JDBC

8.4.1 Código

```
private static final String URL =  
    "jdbc:mysql://" + HOST + ":" + PORTA + "/" + BANCO +  
    "?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true&characterEncoding=UTF-8";
```

8.4.2 Explicação

Esta instrução constrói automaticamente o endereço de ligação ao MySQL.

O resultado final será semelhante a:

```
jdbc:mysql://localhost:3306/aqstore
```

8.4.3 Parâmetros Utilizados

8.4.3.1 useSSL=false

```
useSSL=false
```

Desativa a utilização de SSL.

Como a aplicação funciona numa rede local, não é necessário utilizar certificados de segurança.

8.4.3.2 serverTimezone=UTC

```
serverTimezone=UTC
```

Define o fuso horário utilizado pelo MySQL.

Evita erros relacionados com datas e horas.

8.4.3.3 allowPublicKeyRetrieval=true

```
allowPublicKeyRetrieval=true
```

Permite a autenticação em versões recentes do MySQL.

8.4.3.4 `characterEncoding=UTF-8`

`characterEncoding=UTF-8`

Permite guardar corretamente:

- acentos;
- cedilhas;
- caracteres especiais.

Sem esta configuração poderiam ocorrer erros na apresentação dos dados.

8.5 Método de Conexão

8.5.1 Código

```
public static Connection conectar() {
```

8.5.2 Explicação

Este método é responsável por criar e devolver uma ligação ativa à base de dados.

Sempre que uma classe necessita de aceder ao MySQL, chama este método.

8.6 Carregamento do Driver JDBC

8.6.1 Código

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

8.6.2 Explicação

Antes de criar a ligação, o Java precisa carregar o driver MySQL.

Este driver funciona como um tradutor entre:

```
Java
    ><
Driver JDBC
    ><
MySQL
```

Sem o driver, o Java não conseguiria comunicar com a base de dados.

8.7 Criação da Ligação

8.7.1 Código

```
return DriverManager.getConnection(
    URL,
    USER,
```

```
        PASSWORD  
    );
```

8.7.2 Explicação

O método `getConnection()` recebe:

- URL;
- Utilizador;
- Palavra-passe.

Caso os dados estejam corretos, devolve um objeto do tipo:

`Connection`

Que será utilizado para executar comandos SQL.

8.8 Tratamento de Erros

8.8.1 Driver Não Encontrado

8.8.1.1 Código

```
catch (ClassNotFoundException e) {
```

8.8.2 Explicação

Este erro ocorre quando o driver MySQL não está disponível no projeto.

Exemplos:

- Dependência Maven ausente;
- Driver JDBC não instalado;
- Configuração incorreta.

8.8.3 Erro de Ligação

8.8.3.1 Código

```
catch (SQLException e) {
```

8.8.4 Explicação

Este erro pode ocorrer quando:

- O MySQL está desligado;
- O utilizador não existe;
- A palavra-passe está incorreta;
- A base de dados não existe;
- O servidor não está acessível.

8.9 Exemplo de Utilização

8.9.1 Código

```
try (Connection conn = Conexao.conectar()) {  
  
    String sql =  
        "SELECT * FROM usuarios";  
  
}
```

8.9.2 Explicação

Neste exemplo:

1. O sistema obtém uma ligação;
2. Executa uma consulta SQL;
3. Obtém os resultados da tabela `usuarios`.

Esta mesma ligação é utilizada por:

- Autenticacao;
- AQStoreSistema;
- Gestão de Produtos;
- Gestão de Utilizadores;
- Compras do Dia;
- Relatórios.

8.10 Vantagens da Classe Conexao

A implementação desta classe oferece várias vantagens:

8.10.1 Centralização

Todos os parâmetros de ligação encontram-se num único local.

8.10.2 Reutilização

Qualquer módulo pode utilizar a mesma ligação.

8.10.3 Facilidade de Manutenção

Caso seja necessário alterar:

- servidor;
- utilizador;
- base de dados;

A alteração é realizada apenas nesta classe.

8.10.4 Organização

Mantém o código mais organizado e modular.

8.11 Fluxo da Conexão

AQStore

v

Conexao.java

v

Driver JDBC

v

MySQL

v

Base de Dados AQStore

A classe **Conexao** é um dos componentes mais importantes do sistema AQStore, pois estabelece a comunicação entre a aplicação Java e a base de dados MySQL.

Através da utilização de JDBC foi possível criar uma ligação segura, reutilizável e eficiente, permitindo que todas as funcionalidades do sistema possam consultar e manipular os dados armazenados na base de dados.

Sem esta classe não seria possível implementar funcionalidades como:

- Login;
- Gestão de Produtos;
- Gestão de Utilizadores;
- Registo de Vendas;
- Relatórios;
- Compras do Dia.

9 7. Gestao de Produtos (CRUD)

A gestão de produtos é uma das funcionalidades centrais do sistema AQStore, pois permite controlar todos os produtos disponíveis para venda na loja.

O módulo foi desenvolvido seguindo o conceito **CRUD** (Create, Read, Update e Delete), permitindo ao utilizador efetuar todas as operações fundamentais sobre os produtos armazenados na base de dados.

CRUD significa:

Operação	Descrição
Create	Criar novos produtos
Read	Consultar produtos
Update	Atualizar produtos
Delete	Eliminar produtos

A implementação deste módulo encontra-se principalmente na classe **AQStoreSistema**, responsável pela comunicação entre a aplicação e a base de dados MySQL.

9.1 Objetivos do Módulo

O módulo de produtos foi desenvolvido para permitir:

- Cadastrar novos produtos;
- Consultar produtos existentes;
- Atualizar informações;
- Eliminar produtos;
- Disponibilizar produtos para venda;
- Manter os dados organizados na base de dados.

9.2 Estrutura da Tabela Produtos

A tabela responsável por armazenar os produtos possui a seguinte estrutura:

Campo	Tipo
id	INT
nome	VARCHAR
preco	DECIMAL
stock	INT

9.3 CREATE – Inserção de Produtos

O método `cadastrarProduto()` é responsável por realizar o cadastro de um novo produto na base de dados do sistema AQStore. Esta funcionalidade corresponde à operação **Create** do CRUD (Create, Read, Update e Delete), permitindo adicionar novos produtos à tabela **produtos**.

9.4 Código

```
public boolean cadastrarProduto(String nome, double preco, int stock) {
    String sql = "INSERT INTO produtos (nome, preco, stock) VALUES (?, ?, ?)";

    try (Connection conn = Conexao.conectar());
```

```

        PreparedStatement stmt = conn.prepareStatement(sql)) {

    stmt.setString(1, nome.trim());
    stmt.setDouble(2, preco);
    stmt.setInt(3, stock);
    return stmt.executeUpdate() > 0;

} catch (Exception e) {
    e.printStackTrace();
    return false;
}
}

```

9.5 Explicação

Este método recebe três parâmetros:

Parâmetro	Tipo	Descrição
nome	String	Nome do produto
preco	double	Preço unitário do produto
stock	int	Quantidade disponível em stock

O objetivo do método é inserir estes dados na tabela **produtos** da base de dados MySQL.

9.6 Declaração do Método

```
public boolean cadastrarProduto(String nome, double preco, int stock)
```

9.7 Consulta SQL

```

String sql =
"INSERT INTO produtos (nome, preco, stock)
VALUES (?, ?, ?)";

```

9.7.1 Explicação

Esta instrução SQL é responsável por inserir um novo registo na tabela **produtos**.

Os símbolos ? representam parâmetros que serão substituídos pelos valores recebidos pelo método.

Equivalente em SQL:

```

INSERT INTO produtos(nome, preco, stock)
VALUES ('Ração Premium', 3500, 20);

```

A utilização de parâmetros torna a consulta mais segura e evita ataques do tipo **SQL Injection**.

9.8 Ligação à Base de Dados

```
Connection conn = Conexao.conectar();
```

9.8.1 Explicação

Esta instrução estabelece uma ligação entre a aplicação Java e a base de dados MySQL através da classe **Conexao**.

A ligação será utilizada para executar a instrução SQL.

9.9 Criação do PreparedStatement

```
PreparedStatement stmt =  
    conn.prepareStatement(sql);
```

9.9.1 Explicação

O objeto **PreparedStatement** prepara a instrução SQL antes da sua execução.

As principais vantagens são:

- Melhor desempenho;
- Maior segurança;
- Evita SQL Injection;
- Permite reutilizar consultas.

9.10 Associação dos Parâmetros

9.10.1 Nome

```
stmt.setString(1, nome.trim());
```

O método:

```
trim()
```

remove espaços em branco existentes antes ou depois do nome.

Exemplo:

Antes:

```
"  Ração Premium  "
```

Depois:

```
"Ração Premium"
```

9.10.2 Preço

```
stmt.setDouble(2, preco);
```

Define o preço do produto.

Exemplo:

3500.00

9.10.3 Stock

```
stmt.setInt(3, stock);
```

Define a quantidade disponível em stock.

Exemplo:

25 unidades

9.11 Execução da Consulta

```
return stmt.executeUpdate() > 0;
```

9.11.1 Explicação

O método:

```
executeUpdate()
```

Executa a instrução SQL.

O valor devolvido corresponde ao número de linhas afetadas.

Por exemplo:

1

Significa que um produto foi inserido com sucesso.

A condição:

```
> 0
```

Transforma esse resultado num valor booleano.

Resultado	Valor devolvido
1 linha inserida	true
Nenhuma linha inserida	false

9.12 Tratamento de Exceções

```
catch (Exception e)
```


9.12.1 Explicação

Caso ocorra algum erro durante a execução, o sistema entra no bloco `catch`.

Exemplos de erros:

- Falha na ligação à base de dados;
- Base de dados indisponível;
- Erro na instrução SQL;
- Dados inválidos.

9.13 Impressão do Erro

```
e.printStackTrace();
```

Esta instrução apresenta no terminal todas as informações sobre o erro ocorrido, facilitando a identificação e correção de problemas durante o desenvolvimento.

9.14 Retorno em Caso de Erro

```
return false;
```

Caso a operação não seja concluída com sucesso, o método devolve:

```
false
```

Permitindo que a interface gráfica informe o utilizador de que o produto não foi registado.

9.15 Fluxograma do Método

Receber Dados

```
    v
Criar Consulta SQL
```

```
    v
Abrir Ligação MySQL
```

```
    v
Criar PreparedStatement
```

```
    v
Definir Nome
```

```
    v
Definir Preço
```

```
    v
```

Definir Stock

v
Executar INSERT

v
Inserção realizada?

Sim Não

v v
true false

9.16 Exemplo de Funcionamento

Entrada:

Nome: Ração Premium

Preço: 3500

Stock: 25

Consulta executada:

```
INSERT INTO produtos  
(nome, preco, stock)
```

```
VALUES  
( 'Ração Premium', 3500, 25 );
```

Resultado na base de dados:

ID	Nome	Preço	Stock
15	Ração Premium	3500.00	25

9.17 READ – Consulta de Produtos

A operação READ permite visualizar todos os produtos registados.

9.18 Código

```
public List<Object[]> listarProdutos() {  
    List<Object[]> lista = new ArrayList<Object[]>();  
    String sql = "SELECT id, nome, preco, stock FROM produtos ORDER BY nome";  
  
    try {Connection conn = Conexao.conectar();
```

```

        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery() {

            while (rs.next()) {
                lista.add(new Object[]{
                    rs.getInt("id"),
                    rs.getString("nome"),
                    rs.getDouble("preco"),
                    rs.getInt("stock")
                });
            }

        } catch (Exception e) {
            e.printStackTrace();
        }

        return lista;
    }

```

9.19 Explicação

A consulta utilizada é:

```

SELECT * FROM produtos
ORDER BY nome

```

Esta consulta:

- Obtém todos os produtos;
- Organiza os resultados alfabeticamente.

O ciclo:

```
while (rs.next())
```

Permite percorrer todos os registos encontrados.

9.20 Estrutura dos Dados

Cada produto é armazenado num vetor:

```

new Object[]{
    id,
    nome,
    preco,
    stock
}

```

9.21 UPDATE – Atualização de Produtos

A operação UPDATE permite alterar os dados de um produto já existente.

9.22 Código

```
public boolean atualizarProduto(int id, String nome, double preco, int stock) {
    String sql = "UPDATE produtos SET nome = ?, preco = ?, stock = ? WHERE id = ?";

    try (Connection conn = Conexao.conectar();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, nome.trim());
        stmt.setDouble(2, preco);
        stmt.setInt(3, stock);
        stmt.setInt(4, id);
        return stmt.executeUpdate() > 0;

    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}
```

9.23 Explicação

A consulta SQL utilizada é:

```
UPDATE produtos
SET nome=?, preco=?, stock=?
WHERE id=?
```

Esta operação permite:

- Alterar o nome;
- Alterar o preço;
- Alterar o stock;
- Manter o mesmo identificador.

9.24 Exemplo

Antes:

ID: 1
Produto: Ração
Preço: 2500
Stock: 10

Depois:

ID: 1
Produto: Ração Premium
Preço: 3500
Stock: 15

9.25 DELETE – Eliminação de Produtos

A operação DELETE permite remover produtos da base de dados.

9.26 Código

```
public boolean eliminarProduto(int id) {  
  
    String sql =  
        "DELETE FROM produtos WHERE id=?";  
  
    try (Connection conn = Conexao.conectar();  
        PreparedStatement stmt = conn.prepareStatement(sql)) {  
  
        stmt.setInt(1, id);  
  
        return stmt.executeUpdate() > 0;  
    }  
}
```

9.27 Explicação

A consulta SQL utilizada é:

```
DELETE FROM produtos  
WHERE id=?
```

A operação remove permanentemente o produto selecionado.

9.28 Exemplo

Antes:

ID: 3
Produto: Coleira
Preço: 1200
Stock: 10

Depois:

Produto removido.

O registo deixa de existir na base de dados.

9.29 Interface Gráfica da Gestão de Produtos

A gestão de produtos foi integrada na interface JavaFX.

O utilizador pode:

- Inserir novos produtos;
- Consultar produtos;
- Atualizar informações;
- Eliminar produtos.

9.30 Componentes Utilizados

Componente	Função
TextField	Introdução de dados
TableView	Listagem de produtos
Button	Operações CRUD
Alert	Mensagens ao utilizador

9.31 Fluxo de Funcionamento

Utilizador

v

Interface JavaFX

v

AQStoreSistema

v

Conexao

v

MySQL

9.32 Segurança Implementada

Para garantir maior segurança, todas as operações CRUD utilizam:

PreparedStatement

Esta abordagem:

- Evita SQL Injection;
- Melhora o desempenho;
- Facilita a manutenção do código.

9.33 Vantagens da Implementação CRUD

A implementação CRUD trouxe diversos benefícios:

9.33.1 Organização

Os produtos ficam armazenados de forma estruturada.

9.33.2 Facilidade de Atualização

Permite alterar preços e descrições sem modificar código.

9.33.3 Escalabilidade

Novos produtos podem ser adicionados sem limitações.

9.33.4 Integração

Os produtos ficam imediatamente disponíveis para o módulo de vendas.

A implementação do CRUD de produtos permitiu criar um sistema completo de gestão de inventário para a Loja Aquapet.

Através da integração entre JavaFX, JDBC e MySQL foi possível disponibilizar funcionalidades de criação, consulta, atualização e eliminação de produtos, garantindo uma gestão eficiente e organizada dos dados.

Este módulo constitui a base operacional do sistema AQStore, pois todos os processos de venda dependem diretamente dos produtos registrados na base de dados.

10 8. Gestao de Vendas

A gestão de vendas constitui uma das funcionalidades mais importantes do sistema AQStore, pois é através deste módulo que são efetuadas todas as transações comerciais da loja.

Este módulo permite:

- Selecionar produtos;
- Adicionar produtos ao carrinho;
- Definir quantidades;
- Calcular subtotais;
- Calcular o valor total da venda;
- Processar diferentes formas de pagamento;
- Calcular o troco;
- Registrar a venda na base de dados;
- Gerar recibos em PDF.

Todo o processo foi desenvolvido para simular o funcionamento de um sistema de caixa real.

10.1 Objetivos do Módulo

O módulo de vendas foi desenvolvido com os seguintes objetivos:

- Automatizar o processo de faturação;
- Reduzir erros de cálculo;
- Registrar todas as transações realizadas;
- Emitir comprovativos de venda;
- Disponibilizar informações para relatórios futuros.

10.2 Estrutura de Dados Utilizada

Para armazenar temporariamente os produtos selecionados pelo cliente, o sistema utiliza uma lista dinâmica.

10.3 Código

```
List<Object[]> carrinho =  
    new ArrayList<>();
```

10.4 Explicação

A variável `carrinho` funciona como um armazenamento temporário dos produtos da venda.

Cada posição da lista contém:

```
new Object[] {  
    produto,  
    quantidade,  
    preco,  
    total  
}
```

10.5 Estrutura do Carrinho

Posição	Informação
0	Produto
1	Quantidade
2	Preço Unitário
3	Total

10.6 Exemplo

```
new Object[]{  
    "Ração Premium",  
    2,  
    3500.00,  
    7000.00  
}
```

10.7 Adição de Produtos ao Carrinho

Quando o utilizador selecciona um produto e define uma quantidade, o sistema adiciona automaticamente o item ao carrinho.

10.8 Processo

Produto Selecionado

v

Quantidade

v

Cálculo do Total

v

Adicionar ao Carrinho

10.9 Cálculo do Total

O valor total do item é calculado através da fórmula:

$\text{Total} = \text{Quantidade} \times \text{Preço Unitário}$

10.9.1 Exemplo

Quantidade = 3

Preço = 1500

Total = 4500

10.10 Formas de Pagamento

O sistema suporta diferentes formas de pagamento.

10.11 Opções Disponíveis

Forma de Pagamento
Dinheiro
Cartão
PIX

Estas opções são apresentadas ao operador da caixa durante o processo de venda.

10.12 Pagamento em Dinheiro

Quando o cliente paga em dinheiro, o sistema solicita o valor recebido.

10.13 Fórmula Utilizada

Troco = Valor Recebido - Total da Venda

10.13.1 Exemplo

Total da Venda = 7500

Valor Recebido = 10000

Troco = 2500

O troco é calculado automaticamente pelo sistema.

10.14 Finalização da Venda

Após a confirmação dos dados, o sistema executa o processo de finalização.

10.15 Objetivos

- Guardar a venda;
- Guardar os produtos vendidos;
- Gerar recibo;
- Atualizar os relatórios.

10.16 Método Principal

```
public boolean finalizarVenda(
    String usuario,
    String forma,
    Double recebido,
    Double troco,
    List<Object[]> carrinho)
```

10.17 Parâmetros

Parâmetro	Descrição
usuario	Operador que realizou a venda
forma	Forma de pagamento
recebido	Valor recebido
troco	Troco calculado
carrinho	Lista de produtos vendidos

10.18 Registo da Venda

A primeira operação consiste em gravar a venda principal.

10.19 Estrutura da Tabela

Campo
id_venda
data
usuario
forma_pagamento
valor_recebido
troco

10.20 Exemplo

ID Venda: 1001
Data: 2026-06-18
Utilizador: enderson
Forma: Dinheiro
Recebido: 10000
Troco: 2500

10.21 Registo dos Produtos Vendidos

Após criar a venda principal, o sistema grava cada produto vendido na tabela de compras.

10.22 Estrutura

Campo
produto
quantidade

Campo
preco_unitario
total
forma_pagamento
id_venda

10.23 Exemplo

Produto: Ração Premium
 Quantidade: 2
 Preço: 3500
 Total: 7000
 ID Venda: 1001

10.24 Transações na Base de Dados

O sistema utiliza transações para garantir a integridade dos dados.

10.25 Funcionamento

Iniciar Transação

v
 Inserir Venda

v
 Inserir Produtos

v
 Commit

10.26 Commit

Quando todas as operações são concluídas com sucesso:

```
conn.commit();
```

A venda fica definitivamente registrada.

10.27 Rollback

Se ocorrer algum erro:

```
conn.rollback();
```

Todas as operações são canceladas.

Esta abordagem evita inconsistências na base de dados.

10.28 Geração Automática do Recibo

Após a venda ser registada com sucesso, o sistema gera automaticamente um recibo PDF.

10.29 Informações do Recibo

- Nome da loja;
- Operador;
- Forma de pagamento;
- Produtos vendidos;
- Quantidades;
- Totais;
- Valor recebido;
- Troco.

10.30 Pré-visualização do Recibo

Após gerar o PDF, o sistema abre automaticamente uma janela de pré-visualização.

10.31 Benefícios

- Permite verificar o documento;
- Evita erros de impressão;
- Melhora a experiência do utilizador.

10.32 Fluxo Completo da Venda

Selecionar Produto

v

Adicionar ao Carrinho

v

Selecionar Forma de Pagamento

v

Calcular Troco

v

Finalizar Venda

v

Guardar na Base de Dados

v

Gerar Recibo PDF

v

Pré-visualizar Recibo

10.33 Vantagens da Implementação

A implementação do módulo de vendas oferece diversas vantagens:

10.33.1 Rapidez

Permite efetuar vendas em poucos segundos.

10.33.2 Precisão

Reduz erros de cálculo.

10.33.3 Segurança

Utiliza transações na base de dados.

10.33.4 Organização

Mantém o histórico de vendas armazenado.

10.33.5 Integração

Alimenta automaticamente os relatórios do sistema.

O módulo de gestão de vendas representa o núcleo operacional do sistema AQ-Store.

Através da integração entre JavaFX, JDBC, MySQL, OpenPDF e PDFBox, foi possível criar uma solução completa para registo de vendas, cálculo de pagamentos, emissão de recibos e armazenamento permanente das transações.

Este módulo garante rapidez, fiabilidade e organização no processo de faturação da Loja Aquapet.

11 9. Controlo Automatico de Stock

A versão atual do sistema implementa controlo automático de stock.

11.1 Verificar Stock

```
String sqlVerificarStock = "SELECT stock FROM produtos WHERE nome = ? FOR UPDATE";
```

11.2 Atualizar Stock

```
String sqlAtualizarStock = "UPDATE produtos SET stock = stock - ? WHERE nome = ?";
```

11.3 Código

```
for (Object[] item : carrinho) {
    String produto = (String) item[0];
    int quantidade = (Integer) item[1];

    try (PreparedStatement stmtStock = conn.prepareStatement(sqlVerificarStock)) {
        stmtStock.setString(1, produto);
        try (ResultSet rs = stmtStock.executeQuery()) {
            if (!rs.next()) {
                conn.rollback();
                return false;
            }
            int stockAtual = rs.getInt("stock");
            if (quantidade > stockAtual) {
                conn.rollback();
                return false;
            }
        }
    }
}
```

11.4 Explicação

O sistema verifica o stock antes de confirmar a venda. Caso não exista stock suficiente, a operação é cancelada com rollback.

12 10. Compras do Dia

O módulo **Compras do Dia** foi desenvolvido para permitir a consulta rápida das vendas efetuadas numa determinada data.

Esta funcionalidade é especialmente útil para:

- Acompanhar as vendas diárias;
- Verificar o movimento da loja;
- Conferir operações realizadas pelos operadores;
- Identificar vendas específicas;
- Efetuar correções quando necessário.

O módulo encontra-se integrado na interface principal do sistema AQStore e está diretamente ligado à base de dados MySQL.

12.1 Objetivos do Módulo

A funcionalidade Compras do Dia foi criada com os seguintes objetivos:

- Consultar vendas por data;
- Visualizar os produtos vendidos;
- Consultar valores faturados;
- Facilitar auditorias internas;
- Permitir a eliminação de vendas quando autorizado.

12.2 Interface Gráfica

A consulta das vendas é realizada através de uma página específica do sistema.

O utilizador pode:

- Informar uma data;
- Filtrar os resultados;
- Consultar as vendas encontradas;
- Eliminar uma venda (quando permitido).

12.3 Componentes Utilizados

Componente	Função
TextField	Introdução da data
Button	Filtrar dados
TableView	Exibir resultados
Alert	Mensagens ao utilizador

12.4 Pesquisa por Data

A consulta das vendas é realizada através da data indicada pelo utilizador.

12.5 Exemplo

2026-06-18

Após informar a data, o sistema consulta automaticamente a base de dados.

12.6 Fluxo da Consulta

Utilizador

v

Introduz Data

v

Botão Filtrar

v
AQStoreSistema

v
MySQL

v
Resultados

v
TableView

12.7 Consulta na Base de Dados

A pesquisa é realizada utilizando consultas SQL.

12.8 Objetivo

Obter todas as vendas registadas numa determinada data.

12.9 Informações Apresentadas

O sistema apresenta:

Informação
ID da Venda
Produto
Quantidade
Preço Unitário
Total
Forma de Pagamento
Data

12.10 Exemplo de Resultado

ID Venda: 105
Produto: Ração Premium
Quantidade: 2
Preço: 3500.00
Total: 7000.00
Forma: Dinheiro
Data: 2026-06-18

12.11 Estrutura dos Dados

Os dados apresentados são obtidos através da tabela de compras.

12.12 Campos Utilizados

Campo
id_compra
produto
quantidade
preco_unitario
total
forma_pagamento
id_venda

Estes dados encontram-se associados à venda principal através do campo:

id_venda

12.13 Utilização da TableView

Os resultados da consulta são apresentados numa tabela JavaFX.

12.14 Vantagens

- Organização visual dos dados;
- Facilidade de leitura;
- Navegação intuitiva;
- Seleção rápida de registos.

12.15 Eliminação de Vendas

O sistema permite remover vendas registadas.

Contudo, esta funcionalidade encontra-se protegida por níveis de acesso.

12.16 Permissões

Perfil	Eliminar Venda
ADMIN	Sim
GESTOR	Sim
ATENDENTE	Não

12.17 Controlo de Acesso

Antes de eliminar uma venda, o sistema verifica o perfil do utilizador autenticado.

12.18 Lógica Aplicada

ADMIN

Pode eliminar

GESTOR

Pode eliminar

ATENDENTE

Não pode eliminar

12.19 Justificação da Restrição

A remoção de vendas é uma operação crítica.

Permitir que todos os utilizadores eliminem registos poderia causar:

- Perda de informação;
- Inconsistências financeiras;
- Erros de auditoria;
- Problemas de controlo interno.

Por esse motivo, apenas perfis com maior responsabilidade possuem esta permissão.

12.20 Processo de Eliminação

Quando um utilizador autorizado seleciona uma venda:

Selecionar Venda

v

Confirmar Eliminação

v

Executar DELETE

v

Atualizar Tabela

12.21 Atualização Automática

Após a eliminação de uma venda:

- Os dados são removidos da base de dados;
- A tabela é atualizada automaticamente;
- Os resultados permanecem sincronizados.

Isto evita que o utilizador tenha de sair e voltar a entrar no sistema para visualizar as alterações.

12.22 Benefícios do Módulo

A implementação do módulo Compras do Dia trouxe diversas vantagens.

12.23 Controlo Diário

Permite acompanhar as vendas realizadas ao longo do dia.

12.24 Auditoria

Facilita a conferência de operações efetuadas pelos operadores.

12.25 Correção de Erros

Permite remover registos incorretos quando autorizado.

12.26 Organização

Mantém todas as vendas acessíveis através de uma única interface.

12.27 Fluxo Completo

Introduzir Data

v

Consultar Base de Dados

v

Apresentar Resultados

v

Selecionar Venda

v

Eliminar (Se Permitido)

12.28 Integração com Outros Módulos

O módulo Compras do Dia está diretamente ligado a:

- Gestão de Vendas;
- Relatórios;
- Base de Dados MySQL;
- Gestão de Utilizadores.

Qualquer venda efetuada no sistema passa automaticamente a estar disponível para consulta neste módulo.

O módulo Compras do Dia foi desenvolvido para facilitar a consulta e gestão das vendas diárias da Loja Aquapet.

Através da integração com JavaFX e MySQL, foi possível criar uma funcionalidade simples, rápida e eficiente para pesquisa, visualização e controlo das vendas realizadas.

O controlo de permissões implementado garante maior segurança e protege a integridade dos dados armazenados no sistema, assegurando que apenas utilizadores autorizados possam efetuar alterações críticas.

13 11. Relatorios

O módulo de **Relatórios** foi desenvolvido para fornecer informações detalhadas sobre as vendas realizadas na Loja Aquapet.

Através deste módulo, o utilizador consegue acompanhar o desempenho comercial da loja, analisar tendências de vendas e obter informações importantes para apoio à tomada de decisões.

Os relatórios são gerados com base nos dados armazenados na base de dados MySQL e podem ser exportados para diferentes formatos.

13.1 Objetivos do Módulo

O módulo de relatórios foi desenvolvido com os seguintes objetivos:

- Consultar vendas realizadas;
- Analisar faturação;
- Identificar produtos mais vendidos;
- Avaliar formas de pagamento utilizadas;
- Gerar relatórios mensais e anuais;
- Exportar dados para CSV e Excel;
- Apoiar a gestão da loja.

13.2 Critérios de Pesquisa

O sistema permite gerar relatórios utilizando dois filtros principais:

13.3 Ano

Permite consultar todas as vendas realizadas durante um determinado ano.

13.3.1 Exemplo

2026

13.4 Mês

Permite filtrar os resultados para um mês específico.

13.4.1 Exemplo

Janeiro
Fevereiro
Março
...
Dezembro

13.5 Consulta por Ano e Mês

A versão atual do sistema permite combinar os dois filtros.

13.5.1 Exemplo

Ano: 2026
Mês: Junho

Neste caso serão apresentadas apenas as vendas efetuadas durante o mês de Junho de 2026.

Esta abordagem melhora significativamente a precisão dos relatórios.

13.6 Fluxo de Funcionamento

Selecionar Ano

v

Selecionar Mês

v

Gerar Relatório

v
Consulta SQL

v
MySQL

v
Apresentação dos Dados

13.7 Informações Apresentadas

O relatório apresenta diversas informações relacionadas com as vendas.

13.8 Dados Disponíveis

Campo
Produto
Quantidade
Preço Unitário
Total
Forma de Pagamento
Data
Operador

13.9 Exemplo de Resultado

Produto: Ração Premium
Quantidade: 3
Preço Unitário: 3500.00
Total: 10500.00
Forma de Pagamento: Dinheiro
Data: 2026-06-18
Operador: paulo

13.10 Estrutura dos Dados

Os dados apresentados no relatório são obtidos a partir da combinação de duas tabelas:

13.11 Tabela Venda

Responsável por armazenar:

- Data da venda;
- Utilizador;

- Forma de pagamento;
- Valor recebido;
- Troco.

13.12 Tabela Compra

Responsável por armazenar:

- Produto;
- Quantidade;
- Preço unitário;
- Total.

13.13 Apresentação dos Relatórios

Os resultados são exibidos através de uma tabela JavaFX (`TableView`).

13.14 Vantagens

- Fácil visualização;
- Organização dos dados;
- Possibilidade de filtragem;
- Atualização dinâmica.

13.15 Exportação para CSV

O sistema permite exportar os relatórios para ficheiros CSV.

13.16 Objetivo

Permitir:

- Partilha dos dados;
- Análise em Excel;
- Arquivamento de relatórios.

13.17 Estrutura do Ficheiro

Produto;Quantidade;Preço Unitário;Total;Forma Pagamento;Data

13.18 Benefícios

- Compatibilidade com Microsoft Excel;
- Compatibilidade com LibreOffice Calc;
- Facilidade de importação para outros sistemas.

13.19 Exportação para Excel

Além do formato CSV, o sistema também suporta exportação para Excel (.xlsx).

13.20 Biblioteca Utilizada

Apache POI

13.21 Funcionalidades

- Criação automática de folhas Excel;
- Organização dos dados em colunas;
- Compatibilidade com Microsoft Excel;
- Facilidade de análise posterior.

13.22 Cálculo de Totais

O sistema calcula automaticamente:

13.23 Quantidade Total Vendida

Soma das quantidades

13.24 Valor Total Faturado

Soma dos totais das vendas

13.25 Distribuição por Forma de Pagamento

Permite identificar:

- Total pago em dinheiro;
- Total pago por cartão;
- Total pago por PIX.

13.26 Benefícios para a Gestão

A implementação dos relatórios oferece diversas vantagens.

13.27 Controle Financeiro

Permite acompanhar a faturação da loja.

13.28 Apoio à Decisão

Ajuda a identificar:

- Produtos mais vendidos;

- Produtos menos vendidos;
- Tendências de consumo.

13.29 Planeamento

Facilita a gestão de:

- Compras;
- Stock;
- Promoções.

13.30 Integração com Outros Módulos

O módulo de relatórios recebe informações de:

Gestão de Produtos

v

Gestão de Vendas

v

Compras do Dia

v

Relatórios

Desta forma, qualquer venda realizada no sistema passa automaticamente a fazer parte dos relatórios.

13.31 Exemplo Prático

Considere as seguintes vendas:

Produto	Quantidade	Total
Ração Premium	3	10500
Coleira	2	2400
Brinquedo	1	800

O relatório apresentará:

Quantidade Total Vendida: 6

Valor Total Faturado: 13700

13.32 Vantagens da Implementação

A implementação deste módulo trouxe diversos benefícios.

13.32.1 Automatização

Elimina a necessidade de cálculos manuais.

13.32.2 Rapidez

Geração instantânea dos relatórios.

13.32.3 Precisão

Reduz erros de processamento.

13.32.4 Flexibilidade

Permite diferentes critérios de consulta.

13.32.5 Exportação

Facilita o armazenamento e partilha dos resultados.

O módulo de Relatórios constitui uma importante ferramenta de apoio à gestão da Loja Aquapet.

Através da integração com a base de dados MySQL e da utilização de JavaFX, Apache POI e exportação CSV, foi possível criar uma solução eficiente para consulta e análise das vendas realizadas.

Os relatórios fornecem informações fundamentais para o controlo financeiro, acompanhamento das operações e apoio à tomada de decisões estratégicas dentro da organização.

14 12. Geracao de Recibos PDF

A geração automática de recibos é uma das funcionalidades mais importantes do sistema AQStore, pois permite emitir um comprovativo de venda imediatamente após a conclusão de uma transação.

O recibo é gerado em formato PDF, possibilitando:

- Impressão do comprovativo;
- Arquivamento digital;
- Controlo das vendas realizadas;
- Entrega ao cliente;
- Consulta posterior das transações.

Esta funcionalidade foi implementada através da classe **ReciboVenda**, utilizando a biblioteca **OpenPDF** para criação dinâmica de documentos PDF.

14.1 Objetivos do Módulo

O módulo de geração de recibos foi desenvolvido com os seguintes objetivos:

- Emitir comprovativos automáticos;
- Reduzir o tempo de atendimento;
- Garantir a rastreabilidade das vendas;
- Disponibilizar um documento profissional para o cliente;
- Facilitar futuras auditorias.

14.2 Biblioteca Utilizada

A geração dos recibos é realizada através da biblioteca:

OpenPDF

Esta biblioteca permite:

- Criar documentos PDF;
- Inserir textos formatados;
- Criar tabelas;
- Aplicar estilos;
- Guardar documentos automaticamente.

14.3 Estrutura da Classe ReciboVenda

A classe responsável pela geração dos recibos é:

```
public class ReciboVenda
```

O seu principal objetivo é receber os dados da venda e criar um ficheiro PDF contendo todas as informações necessárias.

14.4 Pasta de Armazenamento

Os recibos são guardados automaticamente numa pasta específica.

14.5 Código

```
private static final String PASTA_RECIBOS = "recibos";
```

14.6 Explicação

Todos os documentos PDF gerados pelo sistema são armazenados na pasta:

recibos/

Caso a pasta não exista, o sistema cria-a automaticamente.

14.7 Criação da Pasta

14.8 Código

```
File pasta = new File(PASTA_RECIBOS);

if (!pasta.exists()) {
    pasta.mkdirs();
}
```

14.9 Explicação

Este bloco verifica se a pasta existe.

Se não existir:

```
pasta.mkdirs();
```

cria automaticamente a estrutura necessária para armazenar os recibos.

14.10 Geração do Nome do Ficheiro

Cada recibo recebe um nome único.

14.11 Código

```
String nomeArquivo =
    "Recibo_" +
    System.currentTimeMillis() +
    ".pdf";
```

14.12 Explicação

O método:

```
System.currentTimeMillis()
```

gera um valor baseado na data e hora atual.

Exemplo:

Recibo_1776513458912.pdf

Desta forma evita-se a duplicação de ficheiros.

14.13 Criação do Documento PDF

14.14 Código

```
Document document = new Document();
```

```
PdfWriter.getInstance(  
    document,  
    new FileOutputStream(caminho)  
);
```

14.15 Explicação

Nesta etapa:

- É criado um novo documento PDF;
- É definida a localização onde será guardado;
- É estabelecida a ligação entre o documento e o ficheiro.

14.16 Abertura do Documento

14.17 Código

```
document.open();
```

14.18 Explicação

Este método abre o documento para permitir a inserção de conteúdo.

Após esta instrução é possível adicionar:

- textos;
- tabelas;
- títulos;
- totais.

14.19 Definição de Fontes

14.20 Código

```
Font titulo =  
    new Font(  
        Font.HELVETICA,  
        16,  
        Font.BOLD  
    );  
  
Font normal =  
    new Font(  
        Font.HELVETICA,  
        11  
    );
```

14.21 Explicação

Foram criados diferentes estilos de letra para melhorar a apresentação do documento.

14.21.1 Fonte Título

Utilizada para:

AQStore - Loja Aquapet Lda

14.21.2 Fonte Normal

Utilizada para:

- Produtos;
- Quantidades;
- Valores;
- Informações da venda.

14.22 Cabeçalho do Recibo

14.23 Código

```
Paragraph cab =  
    new Paragraph(  
        "AQStore - Loja Aquapet Lda",  
        titulo  
    );  
  
cab.setAlignment(  
    Element.ALIGN_CENTER  
);
```

14.24 Explicação

O cabeçalho identifica a empresa responsável pela venda.

O texto é apresentado:

- Em negrito;
- Centralizado;
- Com tamanho superior ao restante conteúdo.

14.25 Informações da Venda

14.26 Código

```
document.add(  
    new Paragraph(  
        "AQStore - Loja Aquapet Lda",  
        titulo  
    );  
);
```

```

        "Utilizador: " + usuario,
        normal
    )
);

document.add(
    new Paragraph(
        "Forma de Pagamento: "
        + formaPagamento,
        normal
    )
);

```

14.27 Explicação

O recibo apresenta:

- Nome do operador;
- Forma de pagamento utilizada.

Estas informações permitem identificar quem realizou a venda e como foi efetuado o pagamento.

14.28 Criação da Tabela de Produtos

14.29 Código

```

PdfPTable tabela =
    new PdfPTable(4);

```

14.30 Explicação

É criada uma tabela com quatro colunas:

Coluna
Produto
Quantidade
Preço
Total

14.31 Cabeçalhos da Tabela

14.32 Código

```

tabela.addCell(
    new Phrase(

```



```

        "Produto",
        negrito
    )
);

tabela.addCell(
    new Phrase(
        "Qtd",
        negrito
    )
);

```

14.33 Explicação

Define os títulos das colunas que irão organizar os produtos vendidos.

14.34 Inserção dos Produtos

14.35 Código

```

for (Object[] item : carrinho) {

    String produto =
        item[0].toString();

    int qtd =
        (Integer) item[1];

    double preco =
        (Double) item[2];

    double total =
        (Double) item[3];
}

```

14.36 Explicação

O ciclo percorre todos os produtos armazenados no carrinho.

Para cada item são obtidos:

- Produto;
- Quantidade;
- Preço;
- Total.

Estas informações são inseridas na tabela do PDF.

14.37 Cálculo do Total Geral

14.38 Código

```
double totalGeral = 0.0;

totalGeral += total;
```

14.39 Explicação

O sistema soma todos os subtotais dos produtos para obter o valor final da venda.

14.40 Exemplo

Ração = 7000

Coleira = 1200

Brinquedo = 800

Total Geral = 9000

14.41 Valor Recebido e Troco

Quando o pagamento é efetuado em dinheiro, o sistema também apresenta:

14.42 Código

```
document.add(
    new Paragraph(
        "Recebido: "
        + df.format(recebido)
    )
);

document.add(
    new Paragraph(
        "Troco: "
        + df.format(troco)
    )
);
```

14.43 Explicação

Estas informações permitem ao cliente confirmar o valor entregue e o troco recebido.

14.44 Mensagem Final

14.45 Código

```
document.add(  
    new Paragraph(  
        "Obrigado pela preferência!",  
        normal  
    )  
);
```

14.46 Explicação

Mensagem de agradecimento apresentada ao cliente no final do recibo.

14.47 Fecho do Documento

14.48 Código

```
document.close();
```

14.49 Explicação

Finaliza o documento e grava todas as informações no ficheiro PDF.

Após esta operação o recibo fica pronto para ser visualizado ou impresso.

14.50 Exemplo de Recibo

AQStore - Loja Aquapet Lda

Utilizador: heriksandro

Forma de Pagamento: Dinheiro

Produto	Qtd	Preço	Total

Ração Premium	2	3500	7000
Coleira	1	1200	1200

TOTAL: 8200

Recebido: 10000

Troco: 1800

Obrigado pela preferência!

14.51 Benefícios da Implementação

A geração automática de recibos oferece diversas vantagens.

14.51.1 Rapidez

O documento é criado instantaneamente.

14.51.2 Organização

Todos os recibos ficam armazenados digitalmente.

14.51.3 Segurança

Evita perda de informação.

14.51.4 Profissionalismo

Fornece um comprovativo formal ao cliente.

14.51.5 Integração

Funciona automaticamente após cada venda.

14.52 Fluxo Completo

Finalizar Venda

v

Obter Dados do Carrinho

v

Criar PDF

v

Inserir Produtos

v

Calcular Totais

v

Guardar Ficheiro

v

Abrir Pré-visualização

O módulo de geração de recibos PDF permite transformar automaticamente cada venda realizada num documento digital profissional.

A utilização da biblioteca OpenPDF possibilitou criar recibos organizados, contendo todas as informações relevantes da transação, contribuindo para um melhor controle das vendas e para uma maior qualidade do serviço prestado ao cliente.

15 13. Pre-visualizacao de Recibos

Após a geração do recibo em formato PDF, o sistema AQStore apresenta automaticamente uma janela de pré-visualização do documento.

Esta funcionalidade foi implementada para permitir que o operador visualize o recibo antes da sua impressão ou armazenamento, proporcionando uma melhor experiência de utilização e reduzindo possíveis erros.

A pré-visualização foi desenvolvida através da classe `VisualizadorPDFFX`, utilizando a biblioteca **Apache PDFBox** em conjunto com **JavaFX**.

15.1 Objetivos do Módulo

A funcionalidade de pré-visualização foi criada com os seguintes objetivos:

- Visualizar o recibo imediatamente após a venda;
- Evitar a abertura de aplicações externas;
- Confirmar os dados antes da impressão;
- Melhorar a experiência do utilizador;
- Integrar todo o processo de faturação numa única aplicação.

15.2 Biblioteca Utilizada

Para a visualização dos documentos PDF foi utilizada a biblioteca:

Apache PDFBox

Esta biblioteca permite:

- Abrir ficheiros PDF;
- Ler páginas de documentos;
- Converter páginas PDF em imagens;
- Integrar documentos PDF em aplicações Java.

15.3 Classe Responsável

A funcionalidade encontra-se implementada na classe:

`VisualizadorPDFFX.java`

Esta classe é responsável por:

- Abrir o PDF gerado;
- Converter o conteúdo em imagem;

- Apresentar a imagem numa janela JavaFX;
- Disponibilizar scroll para navegação.

15.4 Abertura do Documento PDF

Após a geração do recibo, o sistema recebe o caminho do ficheiro PDF.

15.5 Código

```
PDDocument document =
    Loader.loadPDF(
        new File(caminhoPdf)
    );
```

15.6 Explicação

A classe:

`PDDocument`

representa um documento PDF carregado em memória.

O método:

`Loader.loadPDF()`

Abre o ficheiro localizado no caminho indicado.

15.7 Exemplo

`recibos/Recibo_1776513458912.pdf`

Após a abertura, o documento fica disponível para processamento.

15.8 Criação do Renderizador

Depois de abrir o documento, o sistema cria um renderizador.

15.9 Código

```
PDFRenderer renderer =
    new PDFRenderer(document);
```

15.10 Explicação

O objeto:

`PDFRenderer`

É responsável por transformar páginas PDF em imagens.

Esta conversão é necessária porque o JavaFX não possui suporte nativo para exibir ficheiros PDF diretamente.

Assim, o sistema converte a página em imagem antes da apresentação.

15.11 Conversão da Página em Imagem

15.12 Código

```
BufferedImage imagem =  
    renderer.renderImageWithDPI(  
        0,  
        150  
    );
```

15.13 Explicação

O método:

`renderImageWithDPI()`

converte uma página PDF numa imagem.

15.13.1 Parâmetros

0

Indica a primeira página do documento.

150

Define a resolução da imagem em DPI (Dots Per Inch).

15.14 Resultado

PDF

v
BufferedImage

A imagem gerada apresenta uma qualidade suficiente para leitura e impressão.

15.15 Conversão para JavaFX

O JavaFX não utiliza diretamente objetos do tipo `BufferedImage`.

Por esse motivo é necessária uma conversão.

15.16 Código

```
Image imagemFx =  
    SwingFXUtils.toFXImage(  
        imagem,  
        null  
    );
```

15.17 Explicação

A classe:

SwingFXUtils

faz a conversão entre:

BufferedImage

v

JavaFX Image

Esta etapa permite que a imagem seja apresentada nos componentes gráficos do JavaFX.

15.18 Criação do ImageView

Após a conversão, a imagem é colocada num componente gráfico.

15.19 Código

```
ImageView imageView =  
    new ImageView(imagemFx);
```

15.20 Explicação

O componente:

ImageView

É responsável por exibir imagens na interface JavaFX.

Neste caso será utilizado para mostrar o recibo.

15.21 Ajuste Automático da Imagem

15.22 Código

```
imageView.setPreserveRatio(true);  
imageView.setFitWidth(700);
```


15.23 Explicação

15.23.1 Preserve Ratio

```
setPreserveRatio(true)
```

Mantém as proporções originais do documento.

15.23.2 Fit Width

```
setFitWidth(700)
```

Define a largura máxima da imagem.

Isto garante que o recibo seja apresentado de forma adequada dentro da janela.

15.24 Utilização de ScrollPane

Alguns recibos podem ser maiores do que a área disponível na janela.

Para resolver este problema foi utilizado um `ScrollPane`.

15.25 Código

```
ScrollPane scrollPane =  
    new ScrollPane(imageView);
```

15.26 Explicação

O componente:

```
ScrollPane
```

Permite deslocar o conteúdo verticalmente e horizontalmente.

15.27 Benefícios

- Melhor navegação;
- Visualização completa do documento;
- Suporte para recibos longos.

15.28 Criação da Janela

Depois de preparar todos os componentes, o sistema cria a janela de visualização.

15.29 Código

```
Scene scene =  
    new Scene(  
        scrollPane,  
        800,
```

```
        600  
    );
```

15.30 Explicação

A cena define:

Parâmetro	Valor
Largura	800
Altura	600

Estes valores garantem uma área confortável para leitura do recibo.

15.31 Configuração do Stage

15.32 Código

```
Stage stage = new Stage();  
  
stage.setTitle(  
    "Pré-visualização do Recibo"  
);  
  
stage.setScene(scene);
```

15.33 Explicação

O objeto:

Stage

representa uma nova janela JavaFX.

A janela recebe:

- Título;
- Conteúdo;
- Dimensões.

15.34 Exibição da Janela

15.35 Código

```
stage.show();
```

15.36 Explicação

Este método torna a janela visível ao utilizador.

A partir deste momento o recibo pode ser visualizado.

15.37 Fluxo Completo da Pré-visualização

Venda Finalizada

v
Recibo PDF Gerado

v
Abrir Documento

v
Converter PDF em Imagem

v
Converter para JavaFX

v
Criar ImageView

v
Inserir em ScrollPane

v
Mostrar Janela

15.38 Benefícios da Implementação

A funcionalidade de pré-visualização oferece várias vantagens.

15.39 Integração

Todo o processo ocorre dentro da aplicação.

15.40 Rapidez

O recibo é apresentado imediatamente após a venda.

15.41 Facilidade de Utilização

Não é necessário abrir programas externos.

15.42 Melhor Experiência do Utilizador

O operador consegue confirmar os dados antes de imprimir.

15.43 Organização

Permite verificar rapidamente o documento gerado.

15.44 Exemplo de Funcionamento

Finalizar Venda

v
Gerar PDF

v
Abrir Janela

v
Mostrar Recibo

O operador visualiza imediatamente:

- Produtos vendidos;
- Quantidades;
- Totais;
- Forma de pagamento;
- Troco.

A funcionalidade de pré-visualização de recibos representa uma importante melhoria na usabilidade do sistema AQStore.

Através da utilização da biblioteca Apache PDFBox e dos componentes JavaFX foi possível criar uma solução simples, eficiente e totalmente integrada ao processo de faturação.

Esta funcionalidade permite visualizar os recibos gerados sem recorrer a aplicações externas, aumentando a produtividade e a qualidade do atendimento ao cliente.

16 14. Interface Grafica

A interface gráfica do sistema AQStore foi desenvolvida utilizando a biblioteca **JavaFX**, permitindo criar uma aplicação moderna, intuitiva e de fácil utilização.

A escolha do JavaFX deveu-se à sua capacidade de fornecer componentes visuais avançados, boa integração com Java e suporte para interfaces responsivas.

A interface foi projetada para facilitar o trabalho dos operadores da loja, permitindo acesso rápido às funcionalidades principais do sistema.

16.1 Objetivos da Interface

A interface gráfica foi desenvolvida com os seguintes objetivos:

- Facilitar a interação do utilizador com o sistema;
- Organizar as funcionalidades em menus intuitivos;
- Melhorar a produtividade dos operadores;
- Reduzir erros operacionais;
- Apresentar informações de forma clara e organizada.

16.2 Estrutura Geral da Interface

A aplicação está dividida em duas áreas principais:

16.3 Menu Lateral

Responsável pela navegação entre os módulos.

16.4 Área de Conteúdo

Responsável pela apresentação das funcionalidades selecionadas.

16.5 Estrutura Visual

+-----+		
	AQStore	
	+-----+	
	MENU	CONTEÚDO
+-----+		

16.6 Janela de Login

A primeira interface apresentada ao utilizador é a janela de autenticação.

16.7 Código

```
Label lblUsuario = new Label("Usuário:");
Label lblSenha = new Label("Senha");

TextField txtUsuario = new TextField();
PasswordField txtSenha = new PasswordField();
```

```
Button btnEntrar = new Button("Entrar");
```

16.8 Explicação

Nesta interface são utilizados os seguintes componentes:

Componente	Função
Label	Exibir textos
TextField	Receber o utilizador
PasswordField	Receber a senha
Button	Executar login

O componente:

`PasswordField`

Oculta automaticamente os caracteres digitados pelo utilizador.

16.9 Botão de Login

16.10 Código

```
btnEntrar.setOnAction(  
    e -> fazerLogin(stage)  
);
```

16.11 Explicação

Quando o utilizador pressiona o botão **Entrar**, o sistema:

1. Obtém os dados inseridos;
2. Valida os campos;
3. Consulta a base de dados;
4. Verifica as credenciais;
5. Abre a janela principal.

16.12 Janela Principal

Após autenticação bem-sucedida, o sistema apresenta a interface principal.

16.13 Código

```
Stage stage = new Stage();  
  
stage.setTitle("AQStore");
```

```
stage.setScene(scene);  
stage.show();
```

16.14 Explicação

A classe:

Stage

Representa uma janela JavaFX.

Cada janela da aplicação é criada através deste componente.

16.15 Menu Lateral

O menu lateral permite navegar entre os módulos do sistema.

16.16 Código

```
Button btnLoja =  
    criarBotaoMenu(  
        " ",  
        "Loja / Caixa"  
    );  
  
Button btnProdutos =  
    criarBotaoMenu(  
        " ",  
        "Produtos"  
    );  
  
Button btnComprasDia =  
    criarBotaoMenu(  
        " ",  
        "Compras do Dia"  
    );  
  
Button btnRelatorios =  
    criarBotaoMenu(  
        " ",  
        "Relatórios"  
    );
```

16.17 Explicação

Cada botão abre uma funcionalidade específica.

16.17.1 Loja / Caixa

Permite efetuar vendas.

16.17.2 Produtos

Permite gerir produtos.

16.17.3 Compras do Dia

Permite consultar vendas diárias.

16.17.4 Relatórios

Permite gerar relatórios.

16.18 Controlo de Permissões na Interface

A interface adapta-se automaticamente ao perfil do utilizador.

16.19 Código

```
if (isAdmin()) {  
    menu.getChildren().add(btnUsuarios);  
}
```

16.20 Explicação

Apenas utilizadores do tipo:

ADMIN

visualizam o módulo:

Utilizadores

Esta abordagem aumenta a segurança do sistema.

16.21 Utilização do BorderPane

A interface principal utiliza o componente:

```
BorderPane root = new BorderPane();
```

16.22 Explicação

O BorderPane divide a janela em cinco áreas:

TOP

LEFT

CENTER

RIGHT
BOTTOM

No AQStore:

Região	Utilização
LEFT	Menu
CENTER	Conteúdo
TOP	Cabeçalho

16.23 Utilização de VBox

O menu lateral foi criado através de uma VBox.

16.24 Código

```
VBox menu = new VBox(10);
```

16.25 Explicação

A VBox organiza os componentes verticalmente.

Resultado:

Loja
Produtos
Compras do Dia
Relatórios
Utilizadores

16.26 Utilização de HBox

Em várias áreas do sistema foi utilizado o componente:

```
HBox topo = new HBox(10);
```

16.27 Explicação

O HBox organiza elementos horizontalmente.

Exemplo:

[Pesquisar] [Filtrar] [Eliminar]

16.28 Utilização de TableView

Grande parte dos dados do sistema são apresentados através de tabelas.

16.29 Código

```
TableView<Object[]> tabela =  
    new TableView<>();
```

16.30 Explicação

O componente TableView permite:

- Exibir dados;
- Selecionar registros;
- Organizar informações;
- Atualizar conteúdos.

16.31 Exemplo de Utilização

Na gestão de produtos:

ID	Produto	Preço
1	Ração Premium	3500
2	Coleira	1200

16.32 Utilização de Alert

O sistema utiliza caixas de diálogo para comunicar com o utilizador.

16.33 Código

```
Alert alert =  
    new Alert(  
        Alert.AlertType.INFORMATION  
    );
```

16.34 Explicação

Os Alert são utilizados para:

- Informações;
- Avisos;
- Erros;
- Confirmações.

16.35 Exemplo

```
alert.setTitle("Sucesso");  
alert.setContentText(
```

```
        "Produto cadastrado com sucesso."
    );
```

Resultado:

```
+-----+
| Sucesso          |
| Produto cadastrado |
| com sucesso.     |
+-----+
```

16.36 Interface do Carrinho

A área de vendas apresenta os produtos adicionados ao carrinho.

16.37 Estrutura

```
Produto
Quantidade
Preço
Total
```

Os dados são atualizados automaticamente sempre que um novo produto é adicionado.

16.38 Interface dos Relatórios

O módulo de relatórios apresenta:

- Ano;
- Mês;
- Produtos vendidos;
- Quantidades;
- Totais.

Tudo organizado numa tabela dinâmica.

16.39 Interface da Pré-visualização PDF

Após gerar o recibo, o sistema abre uma nova janela.

16.40 Código

```
ImageView imageView =
    new ImageView(imagemFx);

ScrollPane scrollPane =
    new ScrollPane(imageView);
```

16.41 Explicação

O PDF é convertido em imagem e apresentado ao utilizador.

O ScrollPane permite navegar pelo documento.

16.42 Benefícios da Interface Implementada

A interface desenvolvida apresenta diversas vantagens.

16.43 Facilidade de Utilização

Menus simples e intuitivos.

16.44 Organização

Funcionalidades separadas por módulos.

16.45 Rapidez

Acesso rápido às operações principais.

16.46 Segurança

Funcionalidades controladas por nível de acesso.

16.47 Integração

Todos os módulos encontram-se integrados numa única aplicação.

16.48 Fluxo de Navegação

Login

v

Menu Principal

Loja

Produtos

Compras do Dia

Relatórios

Utilizadores

A interface gráfica desenvolvida em JavaFX permitiu criar uma aplicação moderna, organizada e intuitiva para a gestão da Loja Aquapet.

A utilização de componentes como BorderPane, VBox, HBox, TableView, Alert e ScrollPane contribuiu para uma melhor experiência do utilizador, facilitando a navegação e tornando o sistema mais eficiente e profissional.

A integração da interface com os restantes módulos do sistema garantiu uma solução completa para gestão de vendas, produtos, utilizadores e relatórios.

17 15. Implementacao do CRUD

O sistema AQStore implementa o conceito **CRUD (Create, Read, Update e Delete)** para permitir a gestão completa das informações armazenadas na base de dados.

CRUD representa as quatro operações fundamentais utilizadas em praticamente todos os sistemas de informação:

Operação	Significado	Função
Create	Criar	Inserir novos dados
Read	Ler	Consultar dados
Update	Atualizar	Alterar dados existentes
Delete	Eliminar	Remover dados

No AQStore, estas operações são aplicadas principalmente aos módulos de:

- Produtos;
- Utilizadores;
- Compras do Dia;
- Relatórios.

A implementação foi realizada utilizando:

- Java;
- JDBC;
- MySQL;
- JavaFX.

17.1 Objetivos da Implementação CRUD

O CRUD foi desenvolvido para:

- Automatizar a gestão dos dados;
- Facilitar a manutenção da informação;
- Garantir integridade dos registos;
- Permitir operações rápidas e seguras;
- Integrar a aplicação com a base de dados.

17.2 CREATE – Inserção de Dados

A operação CREATE é responsável pela criação de novos registos.

No sistema AQStore esta operação é utilizada principalmente para:

- Cadastrar produtos;
- Criar utilizadores;
- Registar vendas;
- Registar compras.

17.3 Exemplo: Inserção de Produtos

17.3.1 Código

```
public boolean cadastrarProduto(  
    String nome,  
    double preco) {  
  
    String sql =  
        "INSERT INTO produtos(nome, preco, stock) VALUES (?, ?, ?)";  
  
    try (Connection conn = Conexao.conectar();  
        PreparedStatement stmt =  
            conn.prepareStatement(sql)) {  
  
        stmt.setString(1, nome);  
        stmt.setDouble(2, preco);  
        stmt.setDouble(3, stock);  
  
        return stmt.executeUpdate() > 0;  
    }  
}
```

17.4 Explicação

A consulta SQL utilizada é:

```
INSERT INTO produtos(nome, preco, stock)  
VALUES (?, ?, ?)
```

Esta consulta cria um novo registo na tabela de produtos.

O método:

```
stmt.executeUpdate()
```

executa a operação e devolve o número de linhas afetadas.

Se o resultado for superior a zero:

```
return true;
```

significa que o produto foi registado com sucesso.

17.5 Exemplo Prático

Produto: Ração Premium

Preço: 3500

Stock: 20

Resultado:

Produto inserido na base de dados.

17.6 READ – Consulta de Dados

A operação READ permite consultar informações armazenadas na base de dados.

É utilizada em:

- Produtos;
- Utilizadores;
- Compras do Dia;
- Relatórios.

17.7 Exemplo: Consulta de Produtos

17.7.1 Código

```
public List<Object[]> listarProdutos() {  
  
    List<Object[]> lista =  
        new ArrayList<>();  
  
    String sql =  
        "SELECT * FROM produtos ORDER BY nome";
```

17.8 Explicação

A consulta SQL utilizada é:

```
SELECT *  
FROM produtos  
ORDER BY nome
```

Esta consulta recupera todos os produtos registados.

Os dados são armazenados numa lista:

```
List<Object[]>
```

Para posterior apresentação na interface gráfica.

17.9 Processamento dos Resultados

17.9.1 Código

```
while (rs.next()) {  
  
    lista.add(new Object[]{  
        rs.getInt("id"),  
        rs.getString("nome"),  
        rs.getDouble("preco")  
    });  
}
```

17.10 Explicação

O ciclo percorre todos os registos devolvidos pela base de dados.

Cada produto é convertido para:

`Object[]`

e armazenado na lista.

17.11 Resultado

ID	Produto	Preço	Stock
1	Ração Premium	3500	10
2	Coleira	1200	15
3	Brinquedo	800	20

17.12 UPDATE – Atualização de Dados

A operação UPDATE permite alterar informações já existentes.

É utilizada para:

- Atualizar produtos;
- Atualizar utilizadores;
- Alterar preços;
- Alterar níveis de acesso.

17.13 Exemplo: Atualizar Produto

17.13.1 Código

```
public boolean atualizarProduto(  
    int id,  
    String nome,  
    double preco,  
    int stock,) {  
  
    String sql =  
        "UPDATE produtos " +  
        "SET nome=?, preco=?, stock=? " +  
        "WHERE id=";
```

17.14 Explicação

A consulta SQL utilizada é:

```
UPDATE produtos  
SET nome=?, preco=?, stock=?  
WHERE id=?
```

Esta operação procura um produto através do seu ID e altera os respetivos dados.

17.15 Exemplo

Antes:

ID: 1
Produto: Ração
Preço: 2500
Stock: 20

Depois:

ID: 1
Produto: Ração Premium
Preço: 3500
Stock: 25

17.16 DELETE – Eliminação de Dados

A operação DELETE permite remover registos da base de dados.

No AQStore é utilizada em:

- Produtos;
- Utilizadores;

- Compras do Dia.

17.17 Exemplo: Eliminar Produto

17.17.1 Código

```
public boolean eliminarProduto(int id) {  
  
    String sql =  
        "DELETE FROM produtos WHERE id=?";
```

17.18 Explicação

A consulta SQL utilizada é:

```
DELETE FROM produtos  
WHERE id=?
```

Esta operação elimina permanentemente o registo selecionado.

17.19 Exemplo

Antes:

ID: 3
Produto: Brinquedo
Preço: 800

Depois:

Produto removido.

O registo deixa de existir na base de dados.

17.20 Utilização do PreparedStatement

Todas as operações CRUD utilizam:

`PreparedStatement`

17.21 Código

```
PreparedStatement stmt =  
    conn.prepareStatement(sql);
```

17.22 Vantagens

17.22.1 Segurança

Evita SQL Injection.

17.22.2 Desempenho

Melhora a execução das consultas.

17.22.3 Organização

Facilita a manutenção do código.

17.23 Integração com a Interface JavaFX

As operações CRUD estão integradas diretamente na interface gráfica.

17.24 Inserir

Botão Adicionar

17.25 Consultar

TableView

17.26 Atualizar

Botão Atualizar

17.27 Eliminar

Botão Eliminar

17.28 Fluxo de Funcionamento

Utilizador

v

Interface JavaFX

v

AQStoreSistema

v

Conexao

MySQL^v

17.29 Benefícios da Implementação CRUD

A implementação do CRUD trouxe várias vantagens ao sistema.

17.30 Organização

Todos os dados ficam estruturados.

17.31 Facilidade de Manutenção

Permite alterar informações rapidamente.

17.32 Escalabilidade

Novos registos podem ser adicionados sem limitações.

17.33 Integração

Todas as funcionalidades utilizam a mesma lógica.

17.34 Segurança

Utilização de PreparedStatement em todas as operações.

17.35 Aplicação do CRUD nos Módulos

Módulo	Create	Read	Update	Delete
Produtos	V	V	V	V
Utilizadores	V	V	V	V
Compras do Dia	X	V	X	V
Relatórios	X	V	X	X
Vendas	V	V	X	X

17.36 Exemplo Completo

17.37 Produto

17.37.1 Inserir

Ração Premium
3500

17.37.2 Consultar

ID: 1

Produto: Ração Premium

Preço: 3500

17.37.3 Atualizar

Preço: 4000

17.37.4 Eliminar

Produto removido.

A implementação do CRUD permitiu criar uma gestão completa e eficiente dos dados do sistema AQStore.

Através da integração entre Java, JDBC, JavaFX e MySQL foi possível disponibilizar operações de inserção, consulta, atualização e eliminação de forma simples, segura e organizada.

O CRUD constitui a base funcional do sistema, sendo utilizado em praticamente todos os módulos da aplicação e garantindo a correta manipulação das informações armazenadas na base de dados.

18 16. Estruturas de Dados Utilizadas

As estruturas de dados desempenham um papel fundamental no desenvolvimento de qualquer sistema informático, pois são responsáveis pela forma como os dados são organizados, armazenados e manipulados durante a execução do programa.

No desenvolvimento do AQStore foram utilizadas diversas estruturas de dados disponibilizadas pela linguagem Java, sendo a principal delas a **Lista Dinâmica (ArrayList)**.

Estas estruturas permitiram armazenar temporariamente informações como:

- Produtos;
- Carrinho de compras;
- Relatórios;
- Utilizadores;
- Resultados de consultas.

A correta utilização destas estruturas contribuiu para melhorar a organização do código, a eficiência das operações e a integração com a interface gráfica JavaFX.

18.1 Conceito de Estrutura de Dados

Uma estrutura de dados é uma forma organizada de armazenar informações em memória, permitindo:

- Inserção de dados;
- Pesquisa de informações;
- Atualização de registros;
- Remoção de elementos;
- Percorso dos dados armazenados.

A escolha da estrutura adequada influencia diretamente o desempenho e a facilidade de manutenção do sistema.

18.2 Estrutura Principal Utilizada

O AQStore utiliza principalmente a estrutura:

`ArrayList`

A classe `ArrayList` faz parte da biblioteca Java Collections Framework e representa uma lista dinâmica.

18.3 Implementação da Lista Dinâmica

18.4 Código

```
List<Object[]> carrinho =  
    new ArrayList<>();
```

18.5 Explicação

Esta instrução cria uma lista dinâmica denominada:

`carrinho`

responsável por armazenar os produtos selecionados durante uma venda.

A lista pode crescer ou diminuir dinamicamente conforme a necessidade do sistema.

18.6 Funcionamento da Lista

Cada elemento armazenado na lista corresponde a um produto adicionado ao carrinho.

18.7 Estrutura

```
new Object[] {  
    produto,
```

```

        quantidade,
        preco,
        total
    }
}

```

18.8 Organização dos Dados

Posição	Informação
0	Produto
1	Quantidade
2	Preço Unitário
3	Total

18.9 Exemplo

```

new Object[]{
    "Ração Premium",
    2,
    3500.00,
    7000.00
}

```

Neste exemplo:

- Produto = Ração Premium
- Quantidade = 2
- Preço = 3500
- Total = 7000

18.10 Inserção de Elementos

A inserção de produtos na lista é realizada através do método:

18.11 Código

```

carrinho.add(
    new Object[]{
        produto,
        quantidade,
        preco,
        total
    }
);

```

18.12 Explicação

O método:

`add()`

Adiciona um novo elemento ao final da lista.

18.13 Resultado

Antes:

Carrinho vazio

Depois:

[Produto 1]

Após nova inserção:

[Produto 1]

[Produto 2]

18.14 Percurso da Lista

Para percorrer os elementos armazenados utiliza-se um ciclo.

18.15 Código

```
for (Object[] item : carrinho) {  
    }  
}
```

18.16 Explicação

O ciclo percorre todos os elementos presentes na lista.

Cada elemento corresponde a um produto do carrinho.

18.17 Exemplo

Produto 1

Produto 2

Produto 3

O ciclo visitará cada um destes produtos.

18.18 Acesso aos Dados

Os valores são obtidos através das posições do vetor.

18.19 Código

```
String produto =  
    item[0].toString();  
  
int quantidade =  
    (Integer) item[1];  
  
double preco =  
    (Double) item[2];  
  
double total =  
    (Double) item[3];
```

18.20 Explicação

Cada posição contém uma informação específica.

O sistema utiliza estes valores para:

- Gerar recibos;
- Calcular totais;
- Apresentar relatórios;
- Guardar vendas.

18.21 Estruturas Utilizadas nos Relatórios

Os relatórios também utilizam listas dinâmicas.

18.22 Código

```
List<Object[]> relatorio =  
    new ArrayList<>();
```

18.23 Explicação

Cada linha do relatório é armazenada temporariamente numa lista antes de ser apresentada na interface.

18.24 Estruturas Utilizadas na Consulta de Produtos

Os produtos obtidos da base de dados também são armazenados em listas.

18.25 Código

```
List<Object[]> produtos =  
    new ArrayList<>();
```

18.26 Explicação

A lista recebe os dados provenientes da consulta SQL e disponibiliza-os para a interface gráfica.

18.27 Integração com JavaFX

A estrutura ArrayList integra-se facilmente com os componentes JavaFX.

18.28 Exemplo

```
tableView.getItems().addAll(lista);
```

18.29 Explicação

Os dados armazenados na lista são enviados diretamente para a tabela apresentada ao utilizador.

18.30 Porque Não Foi Utilizada uma Pilha?

Uma pilha segue o princípio:

LIFO

Last In First Out

Ou seja:

Último a entrar

Primeiro a sair

18.31 Exemplo de Pilha

Push A

Push B

Push C

Pop

Resultado: C

No AQStore não existe a necessidade de remover sempre o último produto inserido.

Por este motivo a pilha não foi considerada adequada.

18.32 Porque Não Foi Utilizada uma Fila?

Uma fila segue o princípio:

FIFO
First In First Out

Ou seja:

Primeiro a entrar
Primeiro a sair

18.33 Exemplo de Fila

Inserir A
Inserir B
Inserir C

Remover

Resultado: A

No AQStore os produtos podem ser removidos livremente do carrinho, sem respeitar esta ordem.

Por essa razão a fila também não foi adotada.

18.34 Comparação das Estruturas

Estrutura	Característica
Pilha	Último entra, primeiro sai
Fila	Primeiro entra, primeiro sai
ArrayList	Acesso livre aos elementos

18.35 Justificação da Escolha

A utilização de `ArrayList` apresentou várias vantagens:

18.36 Flexibilidade

Permite adicionar e remover elementos em qualquer posição.

18.37 Simplicidade

Implementação simples e intuitiva.

18.38 Integração

Compatibilidade direta com JavaFX.

18.39 Desempenho

Boa performance para listas de dimensão moderada.

18.40 Organização

Facilita a manipulação dos dados.

18.41 Fluxo de Utilização

Produto Selecionado

v

Adicionar à Lista

v

Carrinho

v

Venda

v

Recibo

v

Relatório

18.42 Vantagens da Estrutura Escolhida

A implementação através de listas dinâmicas trouxe diversos benefícios:

18.42.1 Facilidade de Programação

Código mais simples e organizado.

18.42.2 Escalabilidade

Permite trabalhar com qualquer quantidade de produtos.

18.42.3 Manutenção

Facilita futuras alterações ao sistema.

18.42.4 Integração com Base de Dados

Permite armazenar resultados de consultas SQL.

18.42.5 Integração com Interface Gráfica

Facilita a apresentação dos dados em tabelas JavaFX.

18.43 Exemplo Prático

Considere os seguintes produtos:

Ração Premium
Coleira
Brinquedo

Após inserção:

```
carrinho.add(produto1);  
carrinho.add(produto2);  
carrinho.add(produto3);
```

Resultado:

```
[0] Ração Premium  
[1] Coleira  
[2] Brinquedo
```

Todos os elementos ficam disponíveis para:

- consulta;
- atualização;
- remoção;
- exportação.

O AQStore utiliza principalmente a estrutura de dados **Lista Dinâmica (ArrayList)** para armazenar e manipular informações durante a execução do sistema.

A escolha desta estrutura permitiu implementar de forma eficiente funcionalidades como:

- Carrinho de compras;
- Relatórios;
- Consulta de produtos;
- Gestão de utilizadores;
- Processamento de vendas.

Embora conceitos como Pilha e Fila sejam importantes no estudo de Estruturas de Dados, as necessidades específicas do sistema tornaram a utilização de listas dinâmicas a solução mais adequada, oferecendo maior flexibilidade, simplicidade e integração com os restantes módulos da aplicação.

19 17. Controlo de Permissoes

O controlo de permissões é um mecanismo fundamental de segurança implementado no sistema AQStore, responsável por restringir o acesso às funcionalidades de acordo com o perfil de cada utilizador.

Este mecanismo garante que cada utilizador tenha acesso apenas às operações necessárias para desempenhar as suas funções dentro da organização.

A implementação do controlo de permissões contribui para:

- Proteger os dados do sistema;
- Evitar alterações indevidas;
- Reduzir erros operacionais;
- Melhorar a segurança da aplicação;
- Garantir a integridade das informações.

19.1 Objetivos do Controlo de Permissões

O sistema foi desenvolvido para:

- Identificar o perfil do utilizador autenticado;
- Restringir funcionalidades críticas;
- Diferenciar responsabilidades;
- Proteger os dados da base de dados;
- Garantir maior segurança operacional.

19.2 Níveis de Acesso Implementados

O AQStore possui três níveis de acesso distintos:

Nível	Descrição
ADMIN	Administrador do sistema
GESTOR	Responsável operacional
ATENDENTE	Operador de caixa

Cada perfil possui permissões específicas.

19.3 Perfil ADMIN

O perfil ADMIN possui acesso total ao sistema.

19.4 Funcionalidades Disponíveis

- Login;
- Loja / Caixa;
- Gestão de Produtos;

- Compras do Dia;
- Relatórios;
- Gestão de Utilizadores;
- Alteração de Senha;
- Eliminação de Vendas;
- Gestão completa da aplicação.

19.5 Responsabilidades

O administrador é responsável por:

- Gerir utilizadores;
- Definir permissões;
- Corrigir informações;
- Supervisionar o funcionamento do sistema.

19.6 Perfil GESTOR

O perfil GESTOR possui permissões intermédias.

19.7 Funcionalidades Disponíveis

- Loja / Caixa;
- Produtos;
- Compras do Dia;
- Relatórios;
- Alteração de Senha;
- Eliminação de Vendas.

19.8 Restrições

O gestor não possui acesso ao módulo:

Gestão de Utilizadores

Apenas administradores podem criar, atualizar ou remover utilizadores.

19.9 Perfil ATENDENTE

O perfil ATENDENTE possui acesso limitado.

19.10 Funcionalidades Disponíveis

- Loja / Caixa;
- Consulta de Produtos;
- Consulta de Compras;
- Alteração da própria senha.

19.11 Restrições

O atendente não pode:

- Eliminar produtos;
- Criar utilizadores;
- Eliminar utilizadores;
- Aceder aos relatórios administrativos;
- Eliminar vendas.

19.12 Armazenamento das Permissões

O nível de acesso encontra-se armazenado na tabela:

`usuarios`

19.13 Estrutura

Campo
id
username
password
nivel

19.14 Exemplo

ID: 1

Username: admin

Nivel: ADMIN

ID: 2

Username: gestor

Nivel: GESTOR

ID: 3

Username: caixa

Nivel: ATENDENTE

19.15 Identificação do Perfil

Após o login, o sistema cria um objeto da classe `Usuario`.

19.16 Código

```
return new Usuario(  
    rs.getInt("id"),  
    rs.getString("username"),
```



```

        rs.getString("password"),
        rs.getString("nivel")
    );

```

19.17 Explicação

Nesta etapa são carregadas todas as informações do utilizador autenticado.

O campo:

nivel

É utilizado posteriormente para controlar o acesso às funcionalidades.

19.18 Verificação de Permissões

O sistema utiliza métodos específicos para verificar o perfil do utilizador.

19.19 Código

```

private boolean isAdmin() {

    return "ADMIN".equalsIgnoreCase(
        usuarioLogado.getNivelAcesso()
    );
}

```

19.20 Explicação

Este método verifica se o utilizador autenticado possui o perfil:

ADMIN

Se possuir, devolve:

true

Caso contrário:

false

19.21 Verificação do Perfil Gestor

19.22 Código

```

private boolean isGestor() {

    return "GESTOR".equalsIgnoreCase(
        usuarioLogado.getNivelAcesso()
    );
}

```

19.23 Explicação

Este método identifica se o utilizador pertence ao grupo dos gestores.

19.24 Utilização das Permissões

As permissões são verificadas sempre que uma funcionalidade é carregada.

19.25 Exemplo: Gestão de Utilizadores

19.25.1 Código

```
if (isAdmin()) {  
  
    menu.getChildren().add(  
        btnUsuarios  
    );  
}
```

19.25.2 Explicação

Apenas administradores visualizam o botão:

Utilizadores

Os restantes perfis não possuem acesso a esta funcionalidade.

19.26 Exemplo: Compras do Dia

No módulo Compras do Dia apenas administradores e gestores podem eliminar vendas.

19.27 Código

```
if (isAdmin() || isGestor()) {  
  
    topo.getChildren().add(  
        btnEliminar  
    );  
}
```

19.28 Explicação

A condição:

`isAdmin() || isGestor()`

significa:

ADMIN OU GESTOR

Apenas estes perfis conseguem visualizar o botão de eliminação.

19.29 Proteção Adicional

Além de ocultar o botão, o sistema também verifica as permissões durante a execução da ação.

19.30 Código

```
if (!isAdmin() && !isGestor()) {  
  
    mostrarAviso(  
        "Não possui permissão para eliminar vendas."  
    );  
  
    return;  
}
```

19.31 Explicação

Mesmo que um utilizador tente executar a funcionalidade por outro meio, o sistema bloqueia a operação.

Esta abordagem aumenta significativamente a segurança da aplicação.

19.32 Fluxo de Verificação

Login

v
Obter Perfil

v
ADMIN ?

SIM NÃO

v v
Acesso Verificar Gestor
Total

19.33 Exemplo de Funcionamento

19.34 Utilizador ADMIN

Login: admin

Funcionalidades:

V Produtos
V Compras do Dia
V Relatórios
V Utilizadores
V Eliminar Vendas

19.35 Utilizador GESTOR

Login: gestor

Funcionalidades:

V Produtos
V Compras do Dia
V Relatórios
V Eliminar Vendas

X Utilizadores

19.36 Utilizador ATENDENTE

Login: caixa

Funcionalidades:

V Loja
V Consulta de Produtos

X Relatórios
X Utilizadores
X Eliminar Vendas

19.37 Benefícios do Controlo de Permissões

A implementação deste mecanismo trouxe diversas vantagens.

19.38 Segurança

Protege informações sensíveis.

19.39 Organização

Cada utilizador visualiza apenas o necessário.

19.40 Integridade

Reduz alterações indevidas.

19.41 Facilidade de Gestão

Permite controlar responsabilidades.

19.42 Auditoria

Facilita a identificação de operações realizadas.

19.43 Integração com Outros Módulos

O controlo de permissões encontra-se integrado com:

Login

v
Utilizador

v
Controlo de Permissões

Produtos
Compras do Dia
Relatórios
Utilizadores

Desta forma, todas as funcionalidades respeitam as permissões definidas para cada perfil.

O módulo de controlo de permissões é um componente essencial do sistema AQStore, garantindo que cada utilizador tenha acesso apenas às funcionalidades compatíveis com o seu perfil.

Através da implementação de verificações por nível de acesso foi possível criar uma aplicação mais segura, organizada e adequada ao ambiente empresarial.

A utilização dos perfis ADMIN, GESTOR e ATENDENTE permitiu distribuir responsabilidades e proteger operações críticas, contribuindo para a integridade e confiabilidade do sistema.

20 18. Tecnologias Utilizadas

O desenvolvimento do sistema AQStore exigiu a utilização de diversas tecnologias, bibliotecas e ferramentas de software que permitiram implementar as funcionalidades de gestão comercial, autenticação, armazenamento de dados, geração de relatórios e emissão de recibos.

A escolha destas tecnologias teve como principal objetivo garantir:

- Facilidade de desenvolvimento;
- Boa performance;
- Escalabilidade;
- Segurança;
- Integração entre módulos;
- Facilidade de manutenção.

20.1 Linguagem de Programação

20.2 Java

A linguagem principal utilizada no desenvolvimento do sistema foi o **Java**.

20.2.1 Características

- Orientada a Objetos;
- Portável;
- Segura;
- Robusta;
- Multiplataforma.

20.2.2 Aplicação no Projeto

O Java foi utilizado para implementar:

- Interface gráfica;
- Regras de negócio;
- Gestão de utilizadores;
- Gestão de produtos;
- Processamento de vendas;
- Relatórios;
- Geração de recibos.

20.3 Exemplo de Código

```
public class Usuario {  
  
    private int id;  
    private String username;
```

```

    private String password;
    private String nivelAcesso;
}

```

20.3.1 Explicação

A Programação Orientada a Objetos permitiu representar entidades reais através de classes e objetos, tornando o sistema mais organizado e reutilizável.

20.4 Interface Gráfica

20.5 JavaFX

A interface gráfica foi desenvolvida utilizando JavaFX.

JavaFX é uma framework moderna utilizada para criação de aplicações desktop.

20.6 Funcionalidades Utilizadas

- Janelas;
- Menus;
- Botões;
- Tabelas;
- Alertas;
- Formulários;
- Imagens;
- ScrollPane.

20.7 Exemplo

```

Button btnEntrar =
    new Button("Entrar");

```

20.8 Exemplo

```

TableView<Object[]> tabela =
    new TableView<>();

```

20.9 Benefícios

20.9.1 Interface Moderna

Permite criar aplicações visualmente atrativas.

20.9.2 Integração

Integra-se facilmente com Java.

20.9.3 Produtividade

Facilita o desenvolvimento de interfaces complexas.

20.10 Base de Dados

20.11 MySQL

O sistema utiliza MySQL para armazenamento permanente dos dados.

20.12 Dados Armazenados

- Utilizadores;
- Produtos;
- Vendas;
- Compras;
- Relatórios.

20.13 Estrutura da Base de Dados

usuarios
produtos
venda
compra

20.14 Exemplo de Consulta

```
SELECT *  
FROM produtos;
```

20.15 Benefícios

20.15.1 Segurança

Armazenamento confiável.

20.15.2 Organização

Dados estruturados em tabelas.

20.15.3 Escalabilidade

Permite trabalhar com grandes volumes de informação.

20.16 JDBC

20.17 Java Database Connectivity

A comunicação entre Java e MySQL foi realizada através do JDBC.

20.18 Objetivo

Permitir que a aplicação execute:

- SELECT;
- INSERT;
- UPDATE;
- DELETE.

20.19 Exemplo

```
Connection conn =  
    DriverManager.getConnection(  
        URL,  
        USER,  
        PASSWORD  
    );
```

20.20 Explicação

O JDBC cria uma ligação entre a aplicação Java e o servidor MySQL.

20.21 OpenPDF

20.22 Biblioteca de Geração de PDFs

Para a emissão dos recibos foi utilizada a biblioteca:

OpenPDF

20.23 Funcionalidades

- Criar documentos PDF;
- Inserir textos;
- Criar tabelas;
- Exportar documentos.

20.24 Exemplo

```
Document document =  
    new Document();  
  
PdfWriter.getInstance(  
    document,  
    new FileOutputStream(  
        caminho  
    )  
);
```

20.25 Aplicação

Utilizada na classe:

`ReciboVenda.java`

20.26 Apache PDFBox

20.27 Biblioteca de Visualização PDF

A biblioteca PDFBox foi utilizada para abrir e visualizar os recibos gerados.

20.28 Funcionalidades

- Abrir PDFs;
- Ler documentos;
- Converter páginas em imagens;
- Integrar PDFs com JavaFX.

20.29 Exemplo

```
PDDocument document =  
    Loader.loadPDF(  
        new File(caminhoPdf)  
    );
```

20.30 Aplicação

Utilizada na classe:

`VisualizadorPDFFX.java`

20.31 Apache POI

20.32 Biblioteca para Excel

A exportação de relatórios para Excel foi implementada através da biblioteca Apache POI.

20.33 Funcionalidades

- Criar ficheiros XLSX;
- Criar folhas de cálculo;
- Inserir dados em células;
- Exportar relatórios.

20.34 Exemplo

```
Workbook workbook =  
    new XSSFWorkbook();  
  
Sheet sheet =  
    workbook.createSheet(  
        "Relatorio"  
    );
```

20.35 Aplicação

Utilizada no módulo:

Relatórios

20.36 CSV

20.37 Exportação de Dados

O sistema também permite exportar relatórios para o formato CSV.

20.38 Exemplo

```
writer.println(  
    "Produto;Quantidade;"  
    + "Preço;Total"  
);
```

20.39 Benefícios

- Compatibilidade com Excel;
- Compatibilidade com LibreOffice;
- Facilidade de partilha.

20.40 Maven

20.41 Gestão de Dependências

O projeto utiliza Maven para gerir bibliotecas e dependências.

20.42 Benefícios

20.42.1 Automatização

Instala automaticamente as bibliotecas necessárias.

20.42.2 Organização

Centraliza todas as dependências no ficheiro:

`pom.xml`

20.43 Exemplo

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>
    mysql-connector-j
  </artifactId>
  <version>8.3.0</version>
</dependency>
```

20.44 IDE Utilizada

20.45 Visual Studio Code

O desenvolvimento foi realizado utilizando:

Visual Studio Code

20.46 Recursos Utilizados

- Extensão Java;
- Maven;
- Debugger;
- GitHub;
- JavaFX Support.

20.47 Git e GitHub

20.48 Controlo de Versões

O projeto utiliza Git para controlo de versões.

20.49 Benefícios

- Histórico de alterações;
- Recuperação de versões;
- Trabalho colaborativo;
- Backup do código.

20.50 Comandos Utilizados

```
git add .  
git commit -m "Atualização"  
git push
```

20.51 Sistema Operativo

Durante o desenvolvimento foram utilizados:

20.52 Windows 10/11

Ambiente principal de desenvolvimento.

20.53 Tecnologias Integradas

O sistema AQStore integra as seguintes tecnologias:

Java

v

JavaFX

v

JDBC

v

MySQL

v

OpenPDF

v

PDFBox

v

Apache POI

20.54 Tabela Resumo

Tecnologia	Finalidade
Java	Linguagem principal
JavaFX	Interface gráfica
MySQL	Base de dados
JDBC	Ligação à base de dados
OpenPDF	Geração de recibos PDF

Tecnologia	Finalidade
PDFBox	Pré-visualização de PDFs
Apache POI	Exportação para Excel
CSV	Exportação de relatórios
Maven	Gestão de dependências
Git	Controlo de versões
GitHub	Repositório remoto
VS Code	Ambiente de desenvolvimento

20.55 Benefícios da Arquitetura Tecnológica

A combinação destas tecnologias proporcionou:

20.55.1 Modularidade

Separação clara dos componentes.

20.55.2 Escalabilidade

Facilidade de expansão do sistema.

20.55.3 Manutenção

Código mais organizado e reutilizável.

20.55.4 Segurança

Controlo de acessos e utilização de PreparedStatement.

20.55.5 Produtividade

Automatização de diversas tarefas.

O desenvolvimento do AQStore envolveu a integração de várias tecnologias modernas, permitindo criar uma aplicação robusta, segura e eficiente para gestão comercial.

A utilização de Java, JavaFX, MySQL, JDBC, OpenPDF, PDFBox e Apache POI possibilitou implementar funcionalidades completas de autenticação, gestão de produtos, vendas, relatórios e geração de recibos, tornando o sistema adequado para utilização num ambiente empresarial.

A combinação destas ferramentas contribuiu significativamente para a qualidade, desempenho e escalabilidade da solução desenvolvida.

21 19. Resultados Obtidos

O desenvolvimento do sistema AQStore permitiu atingir os objetivos inicialmente definidos para o projeto, resultando numa aplicação funcional capaz de gerir produtos, utilizadores, vendas, relatórios e recibos eletrónicos.

A integração entre JavaFX, MySQL, JDBC e bibliotecas auxiliares possibilitou a criação de uma solução completa para apoio à gestão comercial da Loja Aquapet.

Os resultados obtidos demonstram a aplicação prática dos conhecimentos adquiridos ao longo do curso, especialmente nas áreas de:

- Conceção e Análise de Algoritmos;
- Bases de Dados;
- Interfaces Gráficas;
- Estruturas de Dados;
- Desenvolvimento de Aplicações Desktop.

21.1 Funcionalidades Implementadas

Durante o desenvolvimento do projeto foram implementadas diversas funcionalidades que permitiram alcançar os objetivos propostos.

21.2 Autenticação de Utilizadores

Foi implementado um sistema de autenticação capaz de:

- Validar credenciais;
- Identificar utilizadores;
- Controlar níveis de acesso;
- Restringir funcionalidades.

21.2.1 Resultado

V Funcionalidade concluída

21.3 Gestão de Produtos

Foi implementado o CRUD completo de produtos.

21.3.1 Funcionalidades

- Inserir produtos;
- Consultar produtos;
- Atualizar produtos;
- Eliminar produtos.

21.3.2 Resultado

V CRUD totalmente funcional

21.4 Gestão de Vendas

Foi implementado um sistema de faturação que permite:

- Selecionar produtos;
- Adicionar produtos ao carrinho;
- Calcular totais;
- Processar pagamentos;
- Registrar vendas.

21.4.1 Resultado

V Processo de venda automatizado

21.5 Compras do Dia

Foi criada uma funcionalidade para consulta das vendas diárias.

21.5.1 Funcionalidades

- Pesquisa por data;
- Consulta de vendas;
- Eliminação de vendas (com permissões).

21.5.2 Resultado

V Consulta rápida das operações diárias

21.6 Relatórios

Foi implementado um módulo de relatórios com filtros por:

- Ano;
- Mês.

21.6.1 Funcionalidades

- Consulta de vendas;
- Totais faturados;
- Exportação para CSV;
- Exportação para Excel.

21.6.2 Resultado

V Relatórios dinâmicos e exportáveis

21.7 Recibos PDF

Foi implementada a geração automática de recibos.

21.7.1 Funcionalidades

- Criação automática de PDF;
- Armazenamento dos recibos;
- Identificação das vendas.

21.7.2 Resultado

V Emissão automática de comprovativos

21.8 Pré-visualização de Recibos

Foi implementada uma funcionalidade para visualizar o recibo dentro da própria aplicação.

21.8.1 Funcionalidades

- Abertura automática;
- Conversão PDF para imagem;
- Scroll para navegação.

21.8.2 Resultado

V Visualização integrada no sistema

21.9 Integração dos Módulos

Um dos principais resultados alcançados foi a integração completa entre todos os módulos.

21.10 Fluxo Implementado

Login

v
Produtos

v
Venda

v
Recibo PDF

v

Compras do Dia

v

Relatórios

Todos os módulos comunicam entre si utilizando a mesma base de dados.

21.11 Resultados na Base de Dados

Durante os testes realizados foi possível verificar o correto funcionamento das operações SQL.

21.12 Operações Testadas

Operação	Resultado
INSERT	V
SELECT	V
UPDATE	V
DELETE	V

21.12.1 Resultado

CRUD totalmente operacional

21.13 Resultados da Interface Gráfica

A interface desenvolvida em JavaFX apresentou resultados satisfatórios.

21.14 Características Observadas

- Fácil navegação;
- Menus organizados;
- Resposta rápida;
- Integração com a base de dados;
- Atualização automática dos dados.

21.14.1 Resultado

V Interface funcional e intuitiva

21.15 Resultados do Controlo de Permissões

Foram realizados testes com diferentes tipos de utilizadores.

21.16 Perfis Testados

21.16.1 ADMIN

V Acesso total

21.16.2 GESTOR

V Produtos

V Relatórios

V Compras do Dia

X Gestão de Utilizadores

21.16.3 ATENDENTE

V Loja

V Consulta

X Eliminar vendas

X Gestão de utilizadores

X Relatórios administrativos

21.16.4 Resultado

V Permissões aplicadas corretamente

21.17 Resultados da Exportação

Foram realizados testes de exportação dos relatórios.

21.18 CSV

V Exportação concluída

21.19 Excel

V Exportação concluída

21.20 Compatibilidade

Os ficheiros gerados foram abertos com sucesso em:

- Microsoft Excel;

21.21 Resultados dos Recibos PDF

Foram gerados diversos recibos durante os testes.

21.22 Informações Impressas

- Nome da loja;
- Operador;
- Produtos;
- Quantidades;
- Totais;
- Forma de pagamento;
- Troco.

21.22.1 Resultado

V PDFs gerados corretamente

21.23 Resultados da Pré-visualização

Foram realizados testes com diferentes recibos.

21.23.1 Observações

- PDF carregado corretamente;
- Conversão para imagem concluída;
- Janela aberta automaticamente;
- Scroll funcional.

21.23.2 Resultado

V Visualização sem aplicações externas

21.24 Desempenho do Sistema

Durante os testes realizados, o sistema apresentou um comportamento estável.

21.25 Avaliação

Critério	Resultado
Velocidade	Boa
Estabilidade	Boa
Consumo de Memória	Adequado
Usabilidade	Boa
Integração	Boa

21.26 Benefícios Obtidos

A implementação do AQStore trouxe diversos benefícios.

21.27 Automatização

Redução de tarefas manuais.

21.28 Organização

Melhor gestão das informações.

21.29 Segurança

Controlo de acessos por perfil.

21.30 Rapidez

Processamento automático das vendas.

21.31 Fiabilidade

Registo permanente das operações.

21.32 Aplicação dos Conhecimentos Académicos

O projeto permitiu aplicar conhecimentos adquiridos em várias unidades curriculares.

21.33 Programação

- Java;
- Estruturas de Dados.

21.34 Bases de Dados

- MySQL;
- JDBC;
- SQL.

21.35 Conceção e Análise de Algoritmos

- Modularização;
- Organização de código;
- Boas práticas.

21.36 Interfaces Gráficas

- JavaFX;
- Eventos;
- Componentes visuais.

21.37 Avaliação Geral dos Resultados

Após a conclusão dos testes e validações, verificou-se que o sistema AQStore conseguiu cumprir os objetivos inicialmente definidos.

21.37.1 Funcionalidades Implementadas

V Login
V Produtos
V Vendas
V Compras do Dia
V Relatórios
V PDF
V Pré-visualização
V Permissões
V Exportação

Os resultados obtidos demonstram que o sistema AQStore é uma solução funcional para gestão comercial, permitindo controlar produtos, vendas, utilizadores e relatórios de forma integrada.

A aplicação apresentou um desempenho satisfatório durante os testes realizados, evidenciando a correta utilização das tecnologias selecionadas e a aplicação prática dos conhecimentos adquiridos ao longo do curso de Engenharia Informática.

O projeto cumpriu os objetivos propostos, disponibilizando uma plataforma moderna, organizada e preparada para futuras evoluções.

22 20. Conclusao

O desenvolvimento do sistema **AQStore – Loja Aquapet Lda** permitiu consolidar e aplicar conhecimentos adquiridos ao longo do curso de Engenharia Informática, integrando conceitos da Conceção e Análise de Algoritmos, Bases de Dados, Interfaces Gráficas, Estruturas de Dados e Engenharia de Software num único projeto funcional.

O sistema foi concebido com o objetivo de apoiar a gestão comercial da loja, disponibilizando funcionalidades que permitem controlar produtos, utilizadores, vendas, relatórios e recibos eletrónicos de forma integrada e eficiente.

Ao longo do desenvolvimento foram implementadas diversas funcionalidades essenciais, entre as quais:

- Sistema de autenticação de utilizadores;
- Controlo de permissões por perfil;
- Gestão de produtos através de CRUD;
- Registo e processamento de vendas;
- Consulta de compras do dia;
- Geração de relatórios por ano e mês;
- Exportação para CSV e Excel;
- Geração automática de recibos PDF;
- Pré-visualização dos recibos dentro da aplicação.

A utilização da linguagem **Java**, da framework **JavaFX**, da base de dados **MySQL** e de bibliotecas como **OpenPDF**, **Apache PDFBox** e **Apache POI** permitiu desenvolver uma aplicação moderna, robusta e preparada para futuras evoluções.

Relativamente às estruturas de dados, o sistema fez uso principalmente de **listas dinâmicas (ArrayList)**, utilizadas para armazenar temporariamente informações relacionadas com produtos, carrinhos de compras, vendas e relatórios. Esta escolha permitiu uma implementação simples, flexível e adequada às necessidades do projeto.

Durante os testes realizados verificou-se o correto funcionamento dos módulos implementados, destacando-se:

Funcionalidade	Estado
Login	V Implementado
Gestão de Produtos	V Implementado
Gestão de Utilizadores	V Implementado
Vendas	V Implementado
Compras do Dia	V Implementado
Relatórios	V Implementado
Exportação CSV	V Implementado
Exportação Excel	V Implementado
Recibos PDF	V Implementado
Pré-visualização PDF	V Implementado
Controlo de Permissões	V Implementado

Os resultados obtidos demonstram que os objetivos inicialmente definidos foram alcançados com sucesso, permitindo disponibilizar uma solução funcional para gestão comercial, com uma interface intuitiva e uma estrutura organizada.

Além de resolver um problema real de gestão de vendas e produtos, o projeto proporcionou uma experiência prática no desenvolvimento de aplicações empresariais, permitindo compreender a importância da integração entre software, bases de dados e interfaces gráficas.

Por fim, considera-se que o sistema AQStore possui potencial para futuras melhorias, tais como:

- Dashboard com gráficos estatísticos;
- Backup automático da base de dados;
- Sistema de recuperação de palavra-passe;
- Gestão de fornecedores;
- Gestão de clientes;
- Integração com leitores de código de barras;
- Impressão direta de recibos em impressoras térmicas;
- Disponibilização de uma versão web.

Desta forma, conclui-se que o projeto atingiu os objetivos propostos, constituindo uma solução sólida, organizada e adequada para apoio à gestão comercial da Loja Aquapet Lda, evidenciando a aplicação prática dos conhecimentos adquiridos.